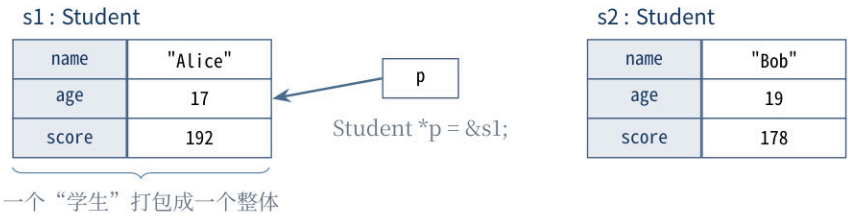
《数据结构与算法》 · C++ 基础 · 第八章

结构体（struct）
基础学习讲义

从“为什么要把数据打包”，到“冒泡排序英雄榜”



本讲义回答的问题

为什么需要结构体？怎样定义、创建和初始化？数组、指针、嵌套、函数参数和 const 怎样与结构体配合？为什么笔记里的代码有几处编译不过？冒泡排序怎样作用在结构体数组上？

全部示例代码均已用 g++ 13（C++17）编译运行验证

排版：XeLaTeX · TikZ / PGFPlots

阅读说明

这份讲义不是把笔记原样誊抄，而是把课堂上写得很简的内容**重新讲透**：先说为什么需要这个知识，再给直观图像，随后是正式定义、语法模板、使用条件、例子、易错点，最后才是练习。你可以按下面的颜色约定快速定位。

颜色 / 样式	含义
深蓝标题栏	核心定义、语法模板、定理——**必须掌握的结论**
蓝灰左线	为什么需要它、直观理解、公式来源
暖米白	示例、学科小史、科普
浅红	易错点、**原图纠错**
浅绿	小结、方法总结、学习自检
教学补全	笔记没写、但自学必须知道的内容（见下）
浅紫	跨学科迁移题的背景

关于“教学补全”的约定 课堂笔记为了节省篇幅，常常省略定义条件、中间步骤和反例。凡是**原笔记没有出现、由本讲义依据标准教材与 cppreference 补充**的内容，都会用左侧青绿线的“教学补全”框标出；凡是**从照片中可靠识别的原始内容**，都在第 3 章逐页列出。两者不会混在一起。笔记里写得不清楚的字，会明确写成“无法可靠辨认”，不会硬猜。

如何使用本讲义 第一次学：按顺序读第 1–8 章，每章末做“学习自检”。检查掌握：做第 9 章的 38 道题（先独立完成，再对答案速查，最后读图解析）。只想查某个语法：直接翻到第 8 章的易错点与辨析表。代码都在 `code/` 目录里有源文件，建议自己敲一遍。

Contents

阅读说明 i

1 本讲学什么：总览与路线 1
- 1.1 本讲要回答的 10 个问题 1
- 1.2 学习路线 1
- 1.3 它在课程体系中的位置 1

2 学科小史与现实应用 2
- 2.1 学科小史：记录（record）是怎样走进编程语言的? 2
- 2.2 从课本到现实：结构体到底有什么用? 3

3 原始笔记：重新识别与纠错 3
- 3.1 逐页内容重新编码（原图可靠识别） 4

3.2 无法可靠辨认的地方 4
3.3 原图纠错 4

4 结构体的定义、创建与初始化 6
4.1 为什么需要结构体 6
4.2 正式定义与语法模板 7
4.3 创建结构体变量的三种方式 8
4.4 访问成员、初始化与整体赋值 9
4.5 教学补全：结构体占多大？（sizeof 与对齐） 10

5 结构体数组与结构体指针 11
5.1 结构体数组 11
5.2 结构体指针 13

6 嵌套结构体、函数参数与 const 16
6.1 结构体嵌套结构体 16
6.2 结构体做函数参数 18
6.2.1 数组做参数：为什么总要多传一个 len？ 20
6.3 const 在结构体中的使用场景 21

7 综合案例：把零件拼成程序 22
7.1 案例一：老师带毕设学生 23
7.2 案例二：英雄冒泡排序 25

8 总结：易错点、概念辨析与知识地图 29
8.1 高频易错点总表 30
8.2 相近概念总对比 30
8.3 基础解题步骤总结 31
8.4 本讲知识地图 31
8.5 后续课程连接 31

9 练习题 32
9.1 知识点—题目对应表 32
9.2 A 基础巩固题（25 题） 32
9.3 B 提升题（10 题） 42
9.4 C 跨学科题（3 题） 49

10 答案速查 52

11 逐题解析与图解 53
11.1 基础题解析（第 1—25 题） 53
11.2 提升题解析（第 26—35 题） 61
11.3 跨学科题解析（第 36—38 题） 66

12 参考资料与说明 **68**

第 1 章　本讲学什么：总览与路线

前面学过的整数、小数、字符、字符串，都只能表示“一个”东西；数组能把“很多个同类东西”排成一排。但现实中的对象往往是**一堆不同类型的信息捆在一起**：一个学生既有姓名（字符串），又有年龄（整数），还有成绩（小数）。本章要解决的，就是“怎样把这些不同类型的信息打包成一个整体”。

本讲一句话概括

结构体（struct）是用户自定义的数据类型，它把若干个可以不同类型的成员，组合成一个整体来创建、存放、传递。

1.1 本讲要回答的 10 个问题

1. 为什么需要结构体?
2. 结构体怎样定义？语法模板是什么?
3. 创建结构体变量有哪三种方式?
4. 怎样访问成员、怎样初始化?
5. 结构体数组怎样放、怎样遍历?
6. 结构体指针与 -> 是什么?
7. 一个结构体里能不能放另一个结构体?
8. 结构体怎样作函数参数？值传递与地址传递差在哪?
9. const 在这里保护什么?
10. 排序算法怎样作用到结构体数组上?

1.2 学习路线

下面的图就是本讲的主线：每一步都在上一步的基础上**多解决一个问题**。

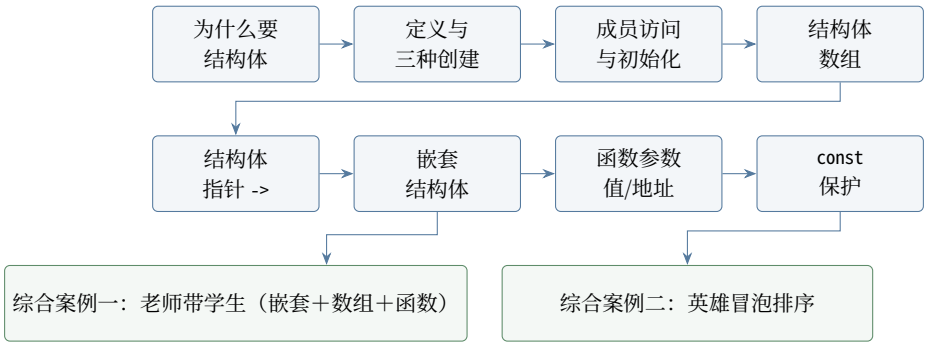


图 1　本讲学习路线：蓝色为语法知识，绿色为综合应用

1.3 它在课程体系中的位置

前置知识 → 本讲 → 后续课程 **前置知识：**基本数据类型（int、float、char）、string、变量与赋值、数组与下标、指针与取地址 &、函数的参数与返回值。

后续课程：类 class（结构体的“加强版”，可以带函数）；《数据结构》中的链表结点、树结点、图的边都是结构体；《算法》中对“带多个字段的记录”按某一字段排序；标准库里的 pair、map 的元素也是类似“打包”的思想；嵌入式与通信中，帧头、传感器记录也常用结构体描述。

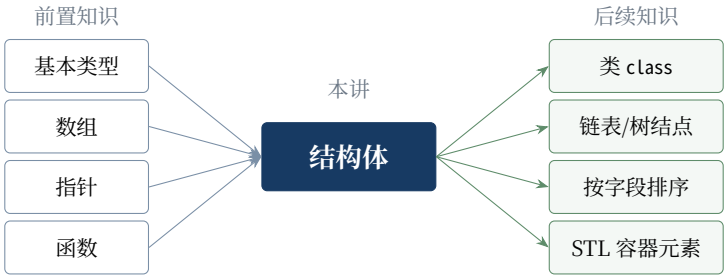


图 2　前置知识与后续知识的位置关系（箭头代表“要先会 → 才能学”）

到这里，我们已经解决了什么？ 知道了本讲要研究“把不同类型的数据打成一个包”，以及整讲的路线。**还没有解决的问题：**这个“包”到底是什么、人们为什么发明它？下一章先用不到两页，回答“它是怎么来的、现实里在哪里用”，然后第 3 章回到你的笔记。

第 2 章　学科小史与现实应用

上一章留下的问题是：“把不同类型的数据打包”这个想法从哪来？这一章很短，目的只有一个：让你知道**人们当年遇到了什么麻烦，才有了今天的 struct**。

2.1 学科小史：记录（record）是怎样走进编程语言的？

当年的麻烦 早期用于科学计算的语言（如 FORTRAN、ALGOL 60 的早期版本）只擅长处理数和同类型的数组。可是商业数据处理要处理的是“职工档案”“订货单”这类**一行里既有文字又有数字**的记录。于是，面向商业数据处理的 COBOL 很早就把“记录”作为语言的一等公民；后来的语言陆续吸收了这个思想，并把它与数组、指针等机制组合起来。

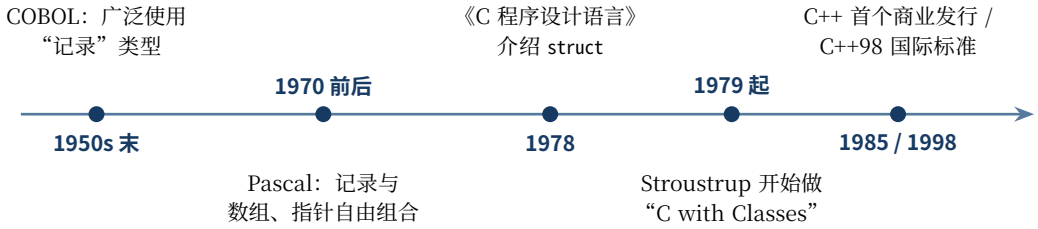


图 3　“记录／结构体”相关的几个关键节点（年份取保守表述）

几个值得记住的节点 **COBOL：**被认为是第一个广泛使用的、支持记录类型的编程语言。**Pascal：**Niklaus Wirth 在 1968–1970 年间设计 Pascal，他回忆说，Pascal 的记录类型来自 COBOL 的记录结构，其具体形式得益于 C. A. R. Hoare 的建议；Pascal 的贡献在于把记录与其他类型组合成**自由嵌套的统一体系**（记录的数组、数组的记录）。**C 与 C++：**C 用 `struct` 提供记录；Stroustrup 在 1979 年开始做“C with Classes”，后来成为 C++。C++ 沿用了 C 的 `struct`，并让它可以包含函数等更多内容，这就是后面你要学的“类”。

为什么今天仍要学它 因为后面所有“复杂数据结构”——链表结点、树结点、图的边、游戏角色、传感器记录——**本质上都是结构体**。不会结构体，就写不出数据结构。

2.2 从课本到现实：结构体到底有什么用？

四个真实的位置 ① **一行数据库记录：**学生表里的一行（学号、姓名、成绩）在程序里就是一个结构体变量，整张表就是结构体数组。② **游戏与机器人的“状态”：**一个角色或一个机械臂关节的位置、速度、血量/角度，常被打包成一个结构体，一次传给更新函数。③ **数据结构课的“结点”：**`struct Node { int val; Node *next; };` 就是链表的一个结点（第 6 章会预告）。④ **固定格式的数据包：**通信协议的帧头、传感器一次采样的数据，常用结构体描述各字段；不过真实工程里还要考虑内存对齐与字节序，这属于后续课程内容。

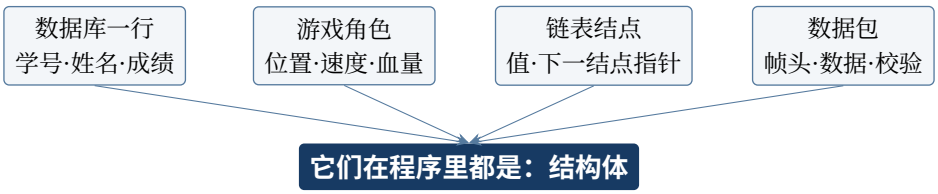


图 4　四种现实对象，同一种程序表示

到这里，我们已经解决了什么？ 知道了结构体是“记录”思想在 C/C++ 中的体现，也知道了它在数据库、游戏、数据结构和通信里的位置。**下一步：**回到你的笔记，先把 13 页手写内容准确识别出来，并指出其中真实存在的编译错误——因为“会写”之前，要先“写对”。

第 3 章　原始笔记：重新识别与纠错

这一章只做一件事：把你 13 页手写笔记（Memo No. 417–429，日期 2026/10/2）**忠实地**整理出来，并用 g++ 实际编译，指出其中真实存在的错误。后面所有章节都会用**纠正后**的代码讲解。

3.1 逐页内容重新编码（原图可靠识别）

页码	小节	笔记内容概要
417	§1 基本概念 §2 定义和使用	结构体属于用户自定义的数据类型，允许用户存储不同的数据类型；语法 `struct 结构体名 {结构体成员列表};`；三种创建方式；示例代码开始（student：name、age、score，变量 s1、s3）
418	§2 续 §3 结构体数组	第二种方法 `s2 = {"Ala",17,182}` 与输出；s3 逐成员赋值；结构体数组的作用与语法 `struct 名 数组名[元素个数] = {{},{},…}`
419	§3 续 §4 结构体指针	数组 `arr[]` 的初始化与 for 遍历输出；红字：还可以给数组中的元素赋值 `arr[2] = {"David",17,201,4};` 结构体指针的作用与“用 -> 访问”的注意
420	§4 续 §5 嵌套	Student 增加 `float score、int grade`；红字“数据类型”；`struct Student *p = &S;` 与 `p->` 输出；嵌套的作用与题意（老师—学生）
421	§5 续	Teacher（id、name、age、Student stu）与 Student 的定义；main 中给 t 及 t.stu 赋值
422	§5 续 §6 函数参数	输出语句；传递方式：值传递、地址传递；Student 增加 hobby；`printStudent(student stu)` 开头
423	§6 续 §7 const	`printStudentaddress(student* stu)`；main 中分别调用并输出 s1.name；红字：不想修改主函数中的数据用值传递，反之用地址传递；const 的作用“防止误操作”
424	§7 续 §8 案例	`void printstudent(const Student* stu)`，红字“加指针可以节约内存，提高速度”；main 调用；案例标题
425	案例一	题意：3 位老师各带 5 名学生；Student（name、score）、Teacher（name、arr[5]）；allocateSpace 开头
426	案例一 续	为名字追加 `nameSeed[i]`；printInfo；main 中 sizeof 求长度
427	案例一 结尾 案例二	案例二题意与期望输出；Hero（name、age、sex）；BubbleSort 开头
428	案例二 续	交换三步；PrintInfo；main 中 5 位英雄的初始化开头
429	案例二 结尾	求 len、调用 BubbleSort、PrintInfo、return 0;

3.2 无法可靠辨认的地方

> **原始资料此处无法可靠辨认**　**(1) 第 419 页**，数组第 2 个元素的最后一个数字，笔迹为“2 3”样式的紧挨数字，**无法可靠辨认**是“2”还是“23”。按 grade（年级）的含义，本讲义取“2”，**以下属于推测，不属于原图可靠识别结果**。
> **(2) 第 419 页**，for 循环输出语句最后一个成员，因笔迹换行被截成 “Sco / ye” 的样子，**无法可靠辨认**；它读起来更像 score（而非 grade）。**可能解释：**笔误，本意是输出 grade。
> 其余笔迹均可辨认。第 429 页下半部分为空白。

3.3 原图纠错

笔记中的代码，我用 g++ 13（`-std=c++17`）逐段编译，得到以下**真实报错**。说明：笔记里手写体的 Void、Using、String 首字母大写，是书写习惯还是笔误无法区分，下面一律按“void、using、string”规范写法处理。

纠错 1　第 418–419 页：缺分号、String　**原始写法：** struct student { String name; int age; int score; int grade; }（右花括号后无分号；类型写成 String）。
存在的问题： g++ 报错 'String' does not name a type、expected ';' after struct definition。C++ 的字符串类型是小写的 string（需要 #include <string>），**结构体定义必须以分号结束**。
规范写法： struct Student { string name; int age; int score; int grade; };
为什么这样改： 结构体定义本身是一条“声明语句”，语句要以 ; 结尾；而且分号后面还可以直接写变量名（第三种创建方式）。
是否影响后续内容： 影响。此处不改，后面所有用到 arr[] 的代码都无法编译。

纠错 2　第 419 页：输出“年级”时写成 score（疑似）　**原始写法（读作）：** <<"年级:"<<arr[i].score。**问题：** 标签是“年级”，却输出了 score，会把分数打印两遍。**规范写法：** <<"年级:"<<arr[i].grade。**注意：** 此处笔迹难辨（见上），按“可能解释”处理。

纠错 3　第 420 页：类型名大小写不一致、&S　**原始写法：** 类型定义为 struct Student，但 main 里写 struct student s1;；指针处写 struct Student *p = &S;。
存在的问题： C++ **区分大小写**，student 与 Student 是两个不同的名字，g++ 报 aggregate ... has incomplete type；同时变量叫 s1，没有叫 S 的变量，报 'S' was not declared。
规范写法： Student s1; Student *p = &s1;
为什么这样改： 取地址运算符 & 后面必须是已经存在的变量名；类型名要与定义处完全一致。
是否影响后续内容： 只影响本页示例，但这是初学者最常见的两类错误。

纠错 4　第 421 页：嵌套顺序错误（Teacher 先于 Student）　**原始写法：** 先写 struct Teacher { …; struct Student stu; };，后写 struct Student { … };。
存在的问题： g++ 报 field 'stu' has incomplete type 'Student'。编译器按**从上到下**读源码，读到 Teacher 里的 Student stu 时，还不知道 Student 有多大，没法给 stu 预留空间。
规范写法：先定义 Student，再定义 Teacher。
为什么这样改： 成员是“结构体本身”（不是指针）时，编译器必须在那一行就知道它的完整大小，详见第 6 章。
是否影响后续内容： 影响，案例一同理。

纠错 5　第 422–424 页：分号、大小写、指针访问用 .、缺引号　**(a)** 第 422 页 struct Student 的右花括号后缺分号。**(b)** 函数形参写 student，类型是 Student（大小写）。**(c)** 第 423 页地址传递函数中，参数是指针 student* stu，函数体第一行写了 stu->name，但输出语句里写成 stu.name、stu.hobby……**指针必须用 ->**，写 . 会报 request for member 'name' in 'stu', which is of pointer type。**(d)** main 里 cout << 姓名: << s1.name 缺少双引号，应为 cout << "姓名:" << s1.name。**(e)** 第 424 页函数定义叫 printstudent，main 里却调用 Printstudent，首字母大小写不同，函数找不到。

纠错 6　第 424 页红字：“加指针可以节约内存、提高速度”　**原始写法：** “加指针可以节约内存，提高速度”。
存在的问题： 方向对，但表述不够准确：节约内存、提高速度的原因是**传地址不再复制整个结构体**（值传递会把整个结构体复制一份）；const 的作用则是**防止通过这个指针误改数据**。两件事是两个机制，不是“加了指针就同时有这两个好处”。（指针本身也要占用空间，在 64 位平台通常是 8 字节。）

规范写法：“传指针（地址）可避免复制整个结构体；再加 const 可防止函数内误修改。”

纠错 7　第 425–426 页：分号、大小写、分数未赋值　Teacher 的右花括号后缺分号；成员写成 struct student arr[5]，类型应为 Student（大小写）；题目要求学生有“考试分数”，allocateSpace 却只给了 name，score 没有赋值（读取未赋值的局部变量，结果不确定）；另外 #include <ctime> 在笔记里没有被用到（原题常用于生成随机分数）。本讲义改为用公式 60 + 10*i + 2*j 给分数赋值，保证结果可复现。

纠错 8　第 427–429 页：形参缺 [] 与 int；内层循环变量；排序方向；题面“结果”　(a) void BubbleSort(struct Hero Array, len)、PrintInfo(struct Hero Array, len)：形参写成了 Array（应为数组形式 Hero Array[]），len 前缺类型 int，g++ 报 'len' has not been declared。
(b) 内层循环写作 for(int i=0; j<len-i-1; j++)：声明的是 i，条件里用的是 j；应为 int j=0。
(c) **比较方向**。笔记写 if (Array[j+1].age > Array[j].age) 才交换。这样“后面比前面大就交换”，大的被换到前面，**结果是降序**，与题目要求的“升序”相反。要升序应写 Array[j].age > Array[j+1].age。
(d) **题面“结果”**。题面最后列出的（“刘备”,23),（“关羽”,22),（“张飞”,20),（“赵云”,21),（“貂蝉”,19) 其实是 main 里数组的**初始顺序**，不是升序排序后的结果。升序后应为：貂蝉 19、张飞 20、赵云 21、关羽 22、刘备 23。
是否影响后续内容：影响。第 7 章案例二采用修正后的写法，并用逐趟图验证。

教学补全：一个可移植性提醒　第 417 页示例只包含了 <iostream>，却使用了 string。在 g++ 下恰好能编译（因为 <iostream> 间接包含了字符串头），但 C++ 标准并不保证。**使用 string 就应写 #include <string>**，后面的笔记页也是这样做的。

到这里，我们已经解决了什么？　笔记内容已完整识别，共发现 8 组真实问题（缺分号、大小写、嵌套顺序、指针访问符、形参缺失、排序方向等）。**接下来：**以纠正后的代码为基础，从“为什么需要结构体”开始，重新把知识一步步讲透。

第 4 章　结构体的定义、创建与初始化

上一章把笔记整理清楚了。从这一章开始，我们用纠正后的代码，从最基本的问题讲起：**为什么需要结构体？**前面的章节只说了“它是一个打包工具”，这一章要让你亲眼看到：不打包会多麻烦。

4.1　为什么需要结构体

先看痛点　假设要记录两名学生。没有结构体时，只能这样写：string name1; int age1; int score1; string name2; int age2; int score2;——**6 个互不相干的变量**。变量多了容易写错，更麻烦的是：要写一个函数“打印一名学生”，就得传 3 个参数；打印两名学生，参数变成 6 个；再来一个学生就再加 3 个。数组能不能解决？不能：数组要求**所有元素类型相同**，一个数组放不下“字符串＋整数＋小数”。

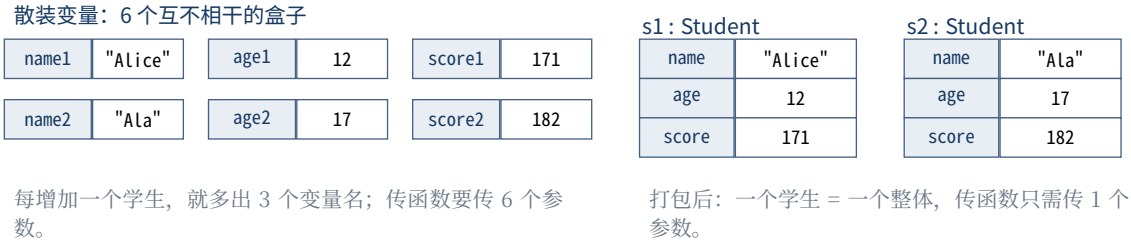


图 5　散装变量与打包成结构体的对比（左：6 个变量；右：2 个结构体变量）

直观理解 结构体就像一张**学生登记卡**：卡上有“姓名”“年龄”“成绩”三栏，每栏填的东西类型不同；不管你有多少个学生，每人一张卡，卡就是“一个整体”。

4.2 正式定义与语法模板

定义：结构体（structure）

结构体是**用户自己定义的数据类型**，它由若干个**成员（member）**组成，成员的类型**可以互不相同**。结构体类型定义好之后，就可以像 `int` 一样用来声明变量、数组、指针和函数参数。

语法模板（笔记 §2 一）

```
struct  结构体名  {  结构体成员列表  };
```

语法成分	含义与注意
`struct`	关键字，告诉编译器“下面要定义一个结构体类型”
结构体名	自己起的类型名（如 `Student`），之后它就是一个类型名；C++ 中创建变量时可以省略前面的 `struct`
`{ ... }`	成员列表。每个成员写法与声明变量相同：`类型 成员名;`
末尾的 `;`	**必须有**。它标志这条类型定义语句结束（也允许在它前面写变量名，见第三种创建方式）

教学补全：使用条件与几个关键事实 (1) 同一个结构体里，成员名不能重复。(2) **定义结构体类型本身不占用内存**，只有**创建变量**时才为各成员分配存储空间（见下图）。(3) 使用 `string` 成员要 `#include <string>`。(4) 成员的个数、类型与顺序一旦写定，后面的初始化就必须与它对应。

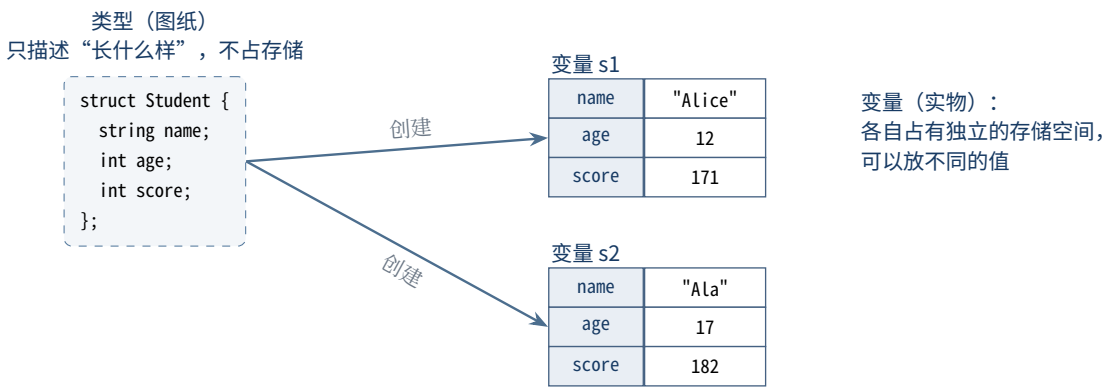


图 6　类型是图纸，变量才是按图纸盖出来的房子

4.3 创建结构体变量的三种方式

笔记的第 2 点给出了三种创建方式，下面把它们放在一起比较（代码已纠正，并用 g++ 验证）：

```cpp
#include <iostream>
#include <string>
using namespace std;

// 方式一：先定义类型，再创建变量，最后逐个成员赋值
struct Student
{
    string name;
    int age;
    int score;
};

int main()
{
    Student s1;                         // 创建变量（方式1）
    s1.name = "Alice";
    s1.age = 12;
    s1.score = 171;

    Student s2 = {"Ala", 17, 182};     // 方式2：创建时用花括号初始化

    struct Student2                     // 方式3：定义类型的同时创建变量 s3
    {
        string name;
        int age;
        int score;
    } s3;
    s3.name = "Ha";
    s3.age = 19;
    s3.score = 171;

    cout << "s1: " << s1.name << " " << s1.age << " " << s1.score << endl;
    cout << "姓名:" << s2.name << " 年龄:" << s2.age << endl;
    cout << "s3: " << s3.name << " " << s3.age << " " << s3.score << endl;
    return 0;
}
```

```
s1: Alice 12 171
姓名:Ala 年龄:17
s3: Ha 19 171
```

方式	写法	特点
1	`Student s1;` 然后 `s1.name = ...;`	先创建、后逐个赋值；创建后成员内容暂时不确定（string 为空串，int 等基本类型不会自动清零）
2	`Student s2 = {"Ala",17,182};`	创建的同时按**成员声明顺序**初始化
3	`struct Student2 {...} s3;`	定义类型的同时创建变量 s3；适合只需要少量变量的场合

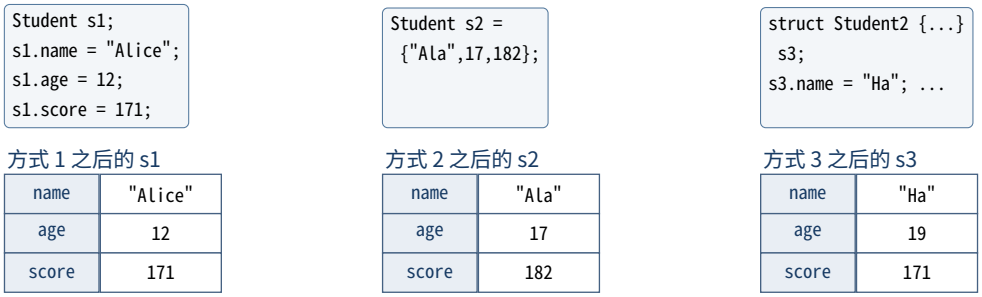


图 7　三种方式殊途同归：最后得到的都是“一个装着三个成员的变量”

易错：忘记末尾分号 / 类型名拼错 `struct Student { ... }`（缺分号）会让编译器把后面的内容当成变量声明的一部分而报错；类型名区分大小写，`Student` 与 `student` 是两个名字（笔记中就出现过，见纠错 3）。

4.4 访问成员、初始化与整体赋值

成员访问运算符 .

对结构体**变量**，用“变量名.成员名”访问某个成员，例如 `s1.age`。它既可以读，也可以写。（对结构体**指针**要用 `->`，见第 5 章。）

花括号初始化的三条规则：

1. **按成员声明顺序一一对应。**`{"Ala",17,182}` 依次给 name、age、score。
2. **给得少，后面的补 0（空串）。**只写前几个值时，没写到的成员被初始化为 0（string 为空串）；`{}` 则全部取默认值。
3. **不允许丢精度的“窄化”转换。**例如 int 成员写 `{3.5}` 会编译报错。

```cpp
#include <iostream>
using namespace std;

struct Point { int x; int y; int z; };

int main()
{
    Point a = {1, 2};          // 只给前两个成员，其余成员被初始化为 0
    Point b = {};              // 全部初始化为 0
    Point c = {3, 4, 5};
    Point d = c;               // 整体赋值：逐个成员拷贝
    d.x = 99;
    cout << a.x << " " << a.y << " " << a.z << endl;
    cout << b.x << " " << b.y << " " << b.z << endl;
    cout << c.x << " " << c.y << " " << c.z << endl;
    cout << d.x << " " << d.y << " " << d.z << endl;
    return 0;
}
```

```
1 2 0
0 0 0
3 4 5
99 4 5
```

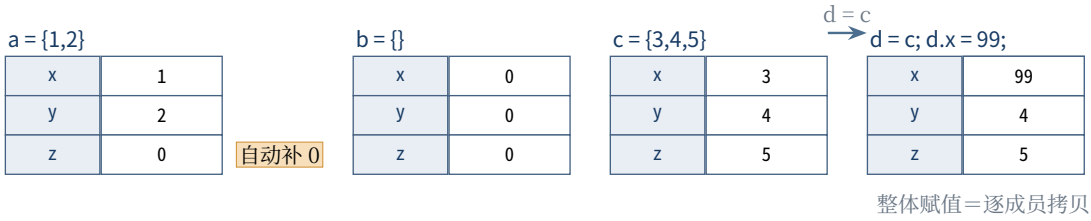


图 8　初始化补 0 与整体赋值：改 d.x 不会影响 c

> **“整体赋值”为什么安全？** `Point d = c;` 会把 c 的每个成员**逐个拷贝**到 d 的对应成员中，d 与 c 之后是两个独立的变量：修改 d.x 只改 d 自己那份，c.x 仍是 3（输出第 3、4 行已验证）。这条性质后面会反复使用：**数组元素之间的交换、值传递的“副本”**，本质上都是整体赋值。

> **正例与反例** **正例：** `Student s = {"Ala", 17, 182};`——顺序与声明一致，类型匹配。
> **反例 1（顺序错）：** `Student s = {17, "Ala", 182};`，g++ 报 `could not convert '17' from 'int' to 'std::string'`。
> **反例 2（窄化）：** `struct S { int a; }; S s = {3.5};`，g++ 报 `narrowing conversion of '3.5e+0' from 'double' to 'int'`。
> **补 0 示例：** `Student a = {"Ala"};` 输出 `[Ala] 0 0`（年龄、分数被补 0）。

> **易错：局部结构体变量不初始化** `Student s1;` 之后，int 成员的值是**不确定**的（笔记里逐个赋值时都赋了值，所以没问题）。只要用到的成员，都要先赋值再读。养成习惯：创建时直接写初始化，或写 `Student s1 = {};`。

4.5 教学补全：结构体占多大？（sizeof 与对齐）

> **教学补全：为什么 sizeof 不是成员大小之和？（拓展，当前阶段只要建立印象）** 案例里会出现 `sizeof(t)/sizeof(t[0])` 求数组个数，所以需要知道 sizeof 对结构体的含义：它返回**整个结构体变量占的字节数**。为了让处理器读写更快，编译器可能在成员之间**插入空白字节（填充）**，所以大小往往大于成员大小之和。cppreference 对此的说法是：“为满足所有成员的对齐要求，可能在某些成员之后插入填充。”

```cpp
#include <iostream>
using namespace std;

struct A { char c; int i; };           // 先 1 字节，再 4 字节
struct B { int i; char c; };
struct C { char c1; int i; char c2; };
struct D { int i; char c1; char c2; };

int main()
{
    cout << "sizeof(A) = " << sizeof(A) << endl;
    cout << "sizeof(B) = " << sizeof(B) << endl;
    cout << "sizeof(C) = " << sizeof(C) << endl;
    cout << "sizeof(D) = " << sizeof(D) << endl;
    return 0;
}
```

```
sizeof(A) = 8
sizeof(B) = 8
sizeof(C) = 12
sizeof(D) = 8
```

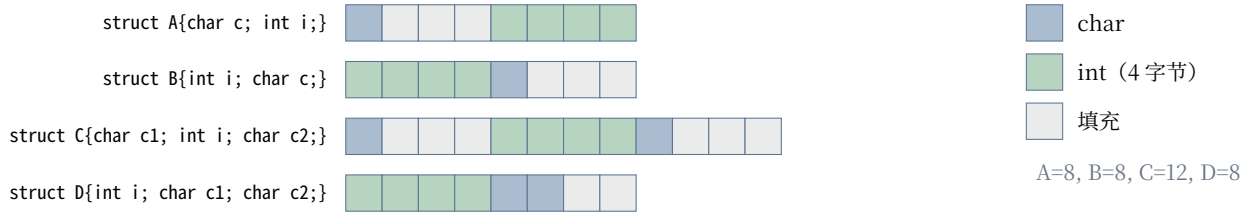


图 9　按成员顺序排列的字节示意（平台：x86-64、g++，int 为 4 字节；数值与平台有关）

直观理解　int 想“从 4 的倍数的位置开始放”。char 放在前面，就得空出 3 字节让 int 对齐；struct C 的 c2 放在最后，为了让结构体整体大小仍是 4 的倍数（这样结构体数组里每个元素的 int 都对齐），末尾又补了 3 字节。把相同类型的成员放一起（比如 D），通常更省空间。

易错　**不要把 sizeof(结构体) 当成成员大小相加**。也不要依赖具体字节数写程序逻辑（不同平台、编译器可能不同）。

学完这里，你应该能够回答

1. 为什么用数组表示“一个学生”不行？结构体解决了什么限制？
2. 写出结构体的语法模板。定义末尾的分号能不能省？
3. “类型”和“变量”哪一个占内存？为什么？
4. 三种创建方式分别是什么？给出各自的一个写法。
5. `Point a = {1,2};` 里 `a.z` 是多少？`Point d = c;` 之后改 `d.x` 会不会改 `c.x`？
6. 结构体 `struct C{char; int; char;}` 为什么是 12 字节？

到这里，我们已经解决了什么？　会定义结构体类型、会用三种方式创建变量、会用 . 访问并初始化成员，也知道整体赋值是逐成员拷贝。**还没有解决的问题：**一个学生有了，但班里有几十个学生，怎样把很多个结构体**成批**管理？怎样“指向”一个结构体而不复制它？这就是下一章：数组与指针。

第 5 章　结构体数组与结构体指针

上一章留下两个问题：班里有几十个学生，怎样**成批**管理？怎样**不复制**就能操作一个结构体？前者的答案是**结构体数组**，后者的答案是**结构体指针**。

5.1　结构体数组

为什么需要它？　把每个学生分别命名成 s1，s2，s3，… 既笨又没法用循环处理。我们已经学过：数组可以用下标配合 for 循环成批处理“同类型”的东西。而结构体变量彼此类型相同，所以**完全可以做数组的元素**。笔记里写的作用就是：“将自定义的结构体放到数组中，方便维护”。

语法模板（笔记 §3.2）

```
struct 结构体名　数组名[元素个数] = { {…}, {…}, …, {…} };
```

外层花括号对应整个数组，里层每一对花括号对应**一个元素（一个结构体）**，里层的值按成员声明顺序给出。元素个数可以省略，由初始化个数推出。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    int age;
    int score;
    int grade;
};

int main()
{
    Student arr[] =
    {
        {"Alice", 17, 192, 3},
        {"Bob",   19, 178, 2},
        {"Xuan",  20, 199, 3}
    };

    arr[2] = {"David", 17, 201, 4};   // 给数组中的某个元素整体赋值

    for (int i = 0; i < 3; i++)
    {
        cout << "姓名:" << arr[i].name << " 年龄:" << arr[i].age
             << " 分数:" << arr[i].score << " 年级:" << arr[i].grade
             << endl;
    }
    return 0;
}
```

```
姓名:Alice 年龄:17 分数:192 年级:3
姓名:Bob 年龄:19 分数:178 年级:2
姓名:David 年龄:17 分数:201 年级:4
```

教学补全：与笔记的两处差别　(1) 笔记中结构体的成员写成 `String`，这里改为 `string`；`for` 循环输出“年级”用 `arr[i].grade`（见纠错 1、2）。(2) 笔记里省略号后面的 `arr[2] = {...}` 是**红字补充**：数组元素可以**整体赋值**，本例把 `Xuan` 那一项整个替换成了 `David`，所以最后输出的是 David（这条红字在 C++11 及以后可以直接使用）。

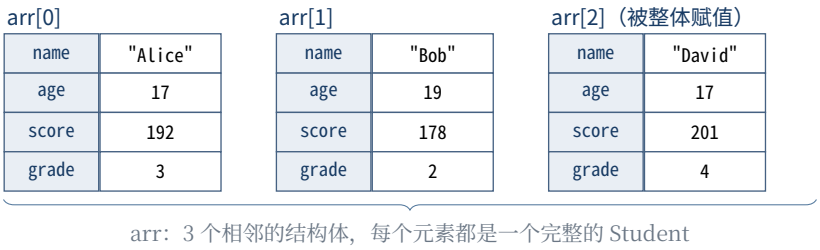


图 10　结构体数组：每个元素是一个结构体，元素在内存中依次相邻

数组下标与成员访问的次序：`arr[i].age` 先用下标`[i]`选出第 `i` 个结构体，再用点号取它的成员——读作“数组第 i 个元素的 age”。

公式：数组元素个数 len（案例中反复出现）

```
int len = sizeof(arr) / sizeof(arr[0]);
```

即　$\text{len} = \dfrac{\text{整个数组占的字节数}}{\text{一个元素占的字节数}}$

公式为什么成立？　数组的 n 个元素类型相同、**首尾相接**，所以整个数组的字节数 $=n\times$ 单个元素的字节数。即 `sizeof(arr)` $=n\cdot$ `sizeof(arr[0])`，两边同除 `sizeof(arr[0])`（它不为 0）即得 $n=$ `sizeof(arr)/sizeof(arr[0])`。
成立条件：`arr` 必须是真正的**数组名**。如果 `arr` 是作为函数形参传进来的，它其实是个指针，这个公式会失效（第 6 章详细讲）。

易错：下标从 0 开始；越界；元素类型　**(1)** 有 3 个元素，合法下标是 `0`，`1`，`2`；`arr[3]` 越界，行为未定义（可能输出垃圾值或崩溃）。**(2)** 循环条件写 `i < 3`（或 `i < len`），不是 `i <= 3`。**(3)** 一个数组里的元素必须是**同一种结构体类型**。

正例与反例　**正例：**`Student arr[] = {{"A",1,2,3},{"B",4,5,6}};` 元素个数由初始化推出为 2。
反例：`Student arr[2] = {{"A",1,2,3},{"B",4,5,6},{"C",7,8,9}};` 初始化项比元素个数多，编译报错 `too many initializers`。

学完这里，你应该能够回答

1. 结构体数组的元素是什么？它们为什么能放进同一个数组？
2. `arr[1].age` 先做哪一步、后做哪一步？
3. 写出求数组元素个数的公式，并说明它为什么成立、什么时候失效。
4. 笔记红字 `arr[2] = {"David",17,201,4};` 的含义是什么？

5.2　结构体指针

为什么需要它？　数组解决了“成批”的问题，但还有一个新问题：结构体可能很大，每次传给函数都复制一份很浪费，而且复制出的副本改了也不会影响原数据。我们想要一个“**指向结构体的路标**”——只记住结构体**在哪里**，需要时顺着路标找过去，这就是指针。笔记里写的作用是：“通过指针访问结构体中的成员”。

结构体指针

结构体指针是一个变量，它保存的是某个结构体变量的**地址**。定义与取地址：`Student *p = &s1;`（`&s1` 是 s1 的地址，让 p 指向 s1）。通过 p 访问成员要用箭头运算符 `->`。

核心等价式（笔记 §4.2）

$$\texttt{p->成员名} \iff \texttt{(*p).成员名}$$

`->` 可读作“箭头指向的那个结构体的成员”。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    int age;
    float score;
    int grade;
};

int main()
{
    Student s1;                 // 创建结构体变量
    s1.name = "Alice";
    s1.age = 19;
    s1.score = 171.82;
    s1.grade = 4;

    Student *p = &s1;           // 指针 p 指向结构体变量 s1

    cout << "姓名:" << p->name << " 年龄:" << p->age
         << " 分数:" << p->score << " 年级:" << p->grade << endl;
    cout << "(*p).age = " << (*p).age << ", 与 p->age 相同" << endl;
    return 0;
}
```

```
姓名:Alice 年龄:19 分数:171.82 年级:4
(*p).age = 19, 与 p->age 相同
```

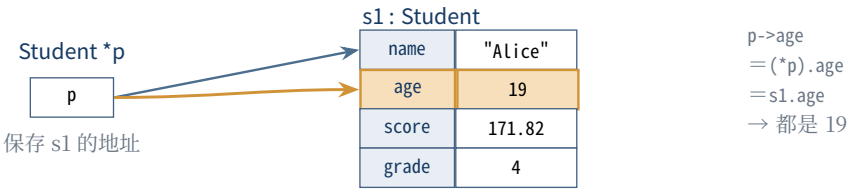


图 11　指针 p 里保存的是 s1 的地址，顺着箭头找到 s1，再取成员

为什么 (*p).age 里的括号不能省？　运算符有优先级：成员访问 `.` 的优先级**高于**解引用 `*`。所以 `*p.age` 会被理解为 `*(p.age)`——先对 p（一个指针）去取成员 age，而指针没有

成员，编译器报错。g++ 报错信息为：`request for member 'age' in 'p', which is of pointer type 'Student*' (maybe you meant to use '->' ?)`。写成 `(*p).age` 才是“先取 p 指向的结构体，再取它的 age”。`->` 就是把这两步合并的简写，cppreference 明确写明：对内建指针类型，`E1->E2` 与 `(*E1).E2` 完全等价。

教学补全：数组名与指针；空指针 (1) 数组名在大多数表达式里可以当作**首元素的地址**，因此可以用指针遍历数组：`q+1` 指向下一个元素。(2) 指针也可以暂时“不指向任何结构体”，此时用 `nullptr` 表示；对空指针访问 `p->成员` 是错误的，使用前先判断。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    int age;
};

int main()
{
    Student arr[3] = {{"A", 17}, {"B", 18}, {"C", 19}};
    Student *q = arr;                   // 数组名可当作“首元素的地址”
    cout << q->age << " " << (q + 1)->age << " " << (q + 2)->name << endl;
    cout << arr[1].age << " " << (*(q + 1)).age << endl;   // 两种写法等价
    return 0;
}
```

```
17 18 C
18 18
```

辨析	s1.age（点号）	p->age（箭头）
左边是什么	结构体**变量**	结构体**指针**
含义	直接取这个变量的成员	先顺着地址找到结构体，再取成员
等价写法	无	`(*p).age`
典型错误	对指针用 `.`	对变量用 `->`（编译报错）

易错：笔记红字“数据类型”的理解；指针不是结构体本身 笔记第 420 页在 student 下面写了红字“数据类型”，意思是：Student（或 struct Student）**是类型名**，不是变量名；s1 才是变量名。`Student *p` 声明的是“指向 Student 的指针”，p 自己只占一个地址的大小（64 位平台通常 8 字节），**并没有**创建新的学生。

学完这里，你应该能够回答

1. 指针里保存的是什么？`&s1` 是什么？
2. `p->age` 等价于什么？为什么 `*p.age` 是错的？

3. 什么时候用 .，什么时候用 ->?
4. 空指针能不能访问成员?

到这里，我们已经解决了什么？ 会用结构体数组成批保存数据，会用 sizeof 求个数，会用指针配合 -> 访问结构体，也知道括号为什么不能省。**还没有解决的问题：**一个结构体里能不能放另一个结构体？结构体作为函数参数时，“复制一份”和“传地址”到底差在哪里？下一章回答这两个问题。

第 6 章　嵌套结构体、函数参数与 const

前两章讲的结构体，成员都是 string、int 这样的“基本”类型。但现实中，一个对象里往往还装着另一个对象：一位老师**辅导一名学生**，老师的信息里就包含学生的信息。同时，我们还要把结构体交给函数处理。本章解决这几个问题。

6.1 结构体嵌套结构体

为什么需要它? 老师有编号、姓名、年龄，还辅导一名学生，学生又有年龄、姓名、分数、年级。如果把学生的所有信息都平铺到老师里面，字段名会冲突（两个人都有 age、name），也看不出“哪些属于学生”。更自然的做法是：**老师结构体里放一个学生结构体成员**。笔记里写的作用就是：“结构体中的成员可以是另一个结构体”。

嵌套结构体

一个结构体的成员，其类型是**另一个结构体类型**，称为嵌套结构体。访问内层成员时，**一层一层用点号**连接：外层变量.内层成员.字段，例如 t.stu.age。

使用条件：被嵌套的类型必须先“完整”定义

作为成员出现的结构体类型，必须在使用它的那一行**之前**已经完整定义。因此应先定义 Student，再定义含有 Student 成员的 Teacher。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student                  // 被嵌套的类型必须先定义
{
    int age;
    string name;
    float score;
    int grade;
};

struct Teacher
{
    int id;
    string name;
```

```
    int age;
    Student stu;                // 成员是另一个结构体
};

int main()
{
    Teacher t;
    t.id = 12458;
    t.name = "Ciman";
    t.age = 72;
    t.stu.age = 17;
    t.stu.name = "Mary";
    t.stu.score = 717.92;
    t.stu.grade = 3;

    cout << "老师姓名:" << t.name << " 老师编号:" << t.id
         << " 老师年龄:" << t.age << " 老师辅导学生姓名:" << t.stu.name
         << " 学生年龄:" << t.stu.age << " 学生分数:" << t.stu.score
         << " 学生年级:" << t.stu.grade << endl;
    return 0;
}
```

```
老师姓名:Ciman 老师编号:12458 老师年龄:72 老师辅导学生姓名:Mary 学生年龄:17 学生分数:717.92 学生年级:3
```

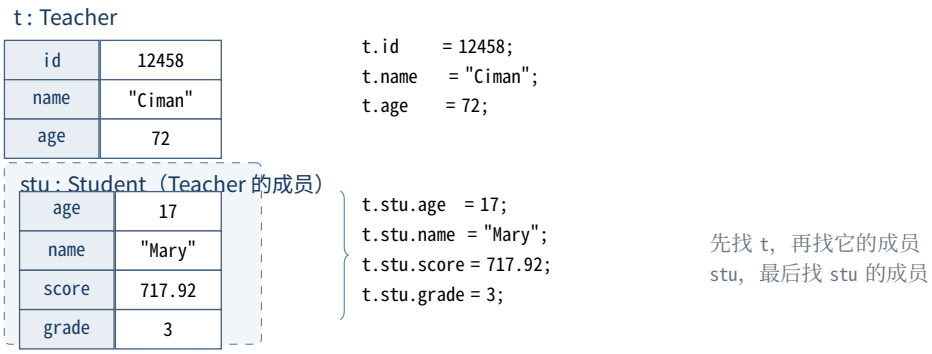


图 12 嵌套结构体在内存中的样子：Student 整个“镶嵌”在 Teacher 里面

> **为什么必须先定义 Student？** 编译器从上往下读代码。读到 Teacher 里的 Student stu; 时，要给 stu 预留空间，就必须知道 Student 到底有多大——也就是它必须已经**完整定义**。如果 Student 还没出现，编译器只知道有这个名字，却不知道大小，g++ 报 field 'stu' has incomplete type 'Student'（第 3 章纠错 4）。

> **教学补全：结构体不能嵌套“自己”，但可以含有指向自己的指针（链表结点的预告）** 如果 struct Bad { int v; Bad next; };，那么 sizeof(Bad) = 4+ sizeof(Bad)，这个方程无解（大小无穷大），所以 g++ 报 field 'next' has incomplete type 'Bad'。**但是指针的大小是固定的**，所以 struct Node { int val; Node *next; }; 是合法的——这正是《数据结构》里**链表结点**的写法。

```
#include <iostream>
using namespace std;

struct Node
```

```
{
    int val;
    Node *next;        // 指向同类型结点的指针（不是 Node 本身）
};

int main()
{
    Node c = {30, nullptr};
    Node b = {20, &c};
    Node a = {10, &b};
    for (Node *p = &a; p != nullptr; p = p->next)
        cout << p->val << " ";
    cout << endl;
    return 0;
}
```

```
10 20 30
```

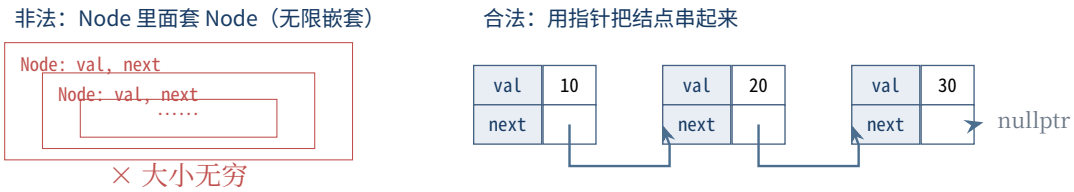


图 13　左：无限套娃，大小无穷（非法）；右：指针大小固定，可以把结点串成链

正例与反例（嵌套）　**正例：** Student 先定义，Teacher 后定义，t.stu.age = 17; 逐层访问。
反例 1： Teacher 在前、Student 在后（第 3 章纠错 4）。
反例 2： t.age = 17; 想给学生赋年龄，却写成了老师的 age——编译通过，但改错了人。内层成员必须经过 stu。

6.2 结构体做函数参数

为什么需要它?　有了结构体，一个学生就是一个“整体”，我们希望函数能直接接收这个整体，而不是接收三四个零散参数。笔记的作用是：“将结构体作为参数向函数中传递”，传递方式有两种：① **值传递**　② **地址传递**。

```
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    string hobby;
    int age;
    int grade;
    float score;
};

// 值传递：形参 stu 是实参的一份副本
```

```cpp
void printStudent(Student stu)
{
    stu.name = "Warma";
    cout << "[值传递]  姓名:" << stu.name << " 爱好:" << stu.hobby
         << " 年龄:" << stu.age << " 年级:" << stu.grade
         << " 分数:" << stu.score << endl;
}

// 地址传递：形参 stu 保存的是实参的地址
void printStudentAddress(Student *stu)
{
    stu->name = "Warma";
    cout << "[地址传递] 姓名:" << stu->name << " 爱好:" << stu->hobby
         << " 年龄:" << stu->age << " 年级:" << stu->grade
         << " 分数:" << stu->score << endl;
}

int main()
{
    Student s1 = {"Bob", "Swimming", 17, 2, 717.28};

    printStudent(s1);
    cout << "main 中 s1.name = " << s1.name << endl;

    printStudentAddress(&s1);
    cout << "main 中 s1.name = " << s1.name << endl;
    return 0;
}
```

```
[值传递]  姓名:Warma 爱好:Swimming 年龄:17 年级:2 分数:717.28
main 中 s1.name = Bob
[地址传递] 姓名:Warma 爱好:Swimming 年龄:17 年级:2 分数:717.28
main 中 s1.name = Warma
```



图 14　值传递改副本，地址传递改原件

选择规则（笔记红字 §6）

不想修改主函数中的数据，用值传递；反之（要修改）用地址传递。

教学补全：规则的两个补充与引用传递　**(1) 值传递的代价：**每次调用都要把整个结构体复制一份，成员多、含有 string 时开销明显。如果既不想改原数据、又不想复制，就用后面的 const 指针。**(2) 笔记第 423 页没有写的第三种方式——引用传递：**形参写成 Student &s，调用时仍写 growUp(a)，函数内用点号。它和地址传递一样能修改原数据，但不必取地址。当前阶段了解即可。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    int age;
};

void growUp(Student &s)      // 引用传递：s 是实参的“别名”，不复制
{
    s.age = s.age + 1;       // 注意：用点号，不用箭头
}

int main()
{
    Student a = {"Ann", 17};
    growUp(a);               // 调用时直接写 a，不用取地址
    cout << a.age << endl;
    return 0;
}
```

```
18
```

> **易错：指针形参里还写 .；传参时忘记取地址；形参写错类型** (1) 地址传递函数里，形参 stu 是指针，成员必须写 stu->name，写 stu.name 编译不过（纠错 5）。(2) 调用地址传递函数要写 printStudentAddress(&s1)，漏了 & 就是把结构体传给指针。(3) 值传递函数里改了形参，别指望 main 里的变量跟着变。

数组做参数：为什么总要多传一个 len?

案例里 allocateSpace(Teacher array[], int len) 除了数组，还要再传一个 len。这是因为：**数组作形参时，传进去的其实只是首元素的地址（指针）**，函数里已经不知道数组有多长，sizeof 也不再是整个数组的大小。

```cpp
#include <iostream>
using namespace std;

struct P { int x; int y; };

void show(P arr[], int len)
{
    cout << "函数内 sizeof(arr) = " << sizeof(arr)
         << "（只是一个指针的大小），len = " << len << endl;
}

int main()
{
    P a[4] = {{1, 2}, {3, 4}, {5, 6}, {7, 8}};
    int len = sizeof(a) / sizeof(a[0]);
    cout << "sizeof(P) = " << sizeof(P) << ", sizeof(a) = " << sizeof(a)
         << ", len = " << len << endl;
```

```
    show(a, len);
    return 0;
}
```

```
sizeof(P) = 8，sizeof(a) = 32，len = 4
函数内 sizeof(arr) = 8（只是一个指针的大小），len = 4
```

（编译时 g++ 还会给出 `-Wsizeof-array-argument` 警告，正是在提醒这件事。）

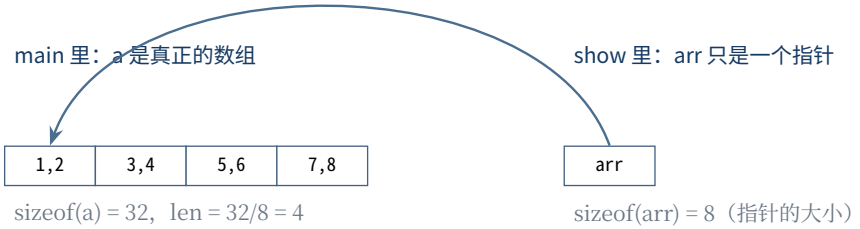


图 15　数组作参数“退化”成指针：元素个数必须单独传

6.3 const 在结构体中的使用场景

为什么需要它？ 前面说：值传递安全但要复制；地址传递不复制但函数里**可能误改**原数据。我们想要一种**“只看不改”**的传递方式——既不复制，又不允许修改。笔记里写的就是：“用 const 来防止误操作”。

指向常量的指针 `const Student *`

形参写成 `const Student *stu`：**通过 stu 只能读，不能写所指结构体的成员**。函数体里若出现 `stu->age = 100;`，编译器直接报错：`assignment of member 'S::a' in read-only object`。

```
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    int age;
    int grade;
    string name;
    float score;
};

// const 指针：只读，函数体内无法通过 stu 修改结构体
void printStudent(const Student *stu)
{
    // stu->age = 100;   // 取消注释会编译报错：assignment of member in read-only object
    cout << "年龄:" << stu->age << " 年级:" << stu->grade
         << " 姓名:" << stu->name << " 分数:" << stu->score << endl;
}

int main()
{
```

```
    Student s1 = {17, 3, "Bob", 717.92};
    printStudent(&s1);
    return 0;
}
```

> 年龄:17 年级:3 姓名:Bob 分数:717.92

教学补全：const 写在不同位置，保护的东西不同（已用 g++ 逐条验证）

声明	含义	能改所指内容?	能改指向?
`const Student *p`	指向常量的指针（笔记用法）	否	是
`Student * const p`	常量指针：指向固定	是	否
`const Student * const p`	两者都固定	否	否

另外，`f(const Student *p)` 既可以接收普通变量的地址，也可以接收 const 变量的地址。

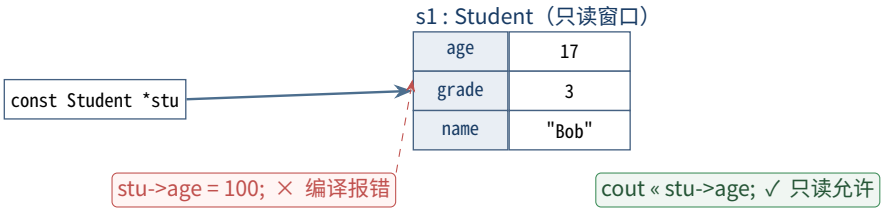


图 16　const 给结构体加了“只读窗口”：读可以，写会被编译器拦下

易错：对 const 与传递方式的几个误解 (1) `const Student *p` 并不要求被指向的变量本身是 const。(2) 在 const 指针函数里写 `p->age = ...` 是**编译错误**，不是运行时错误。(3) 加 const 并不会“让速度更快”，速度的提升来自**不复制整个结构体**（见纠错 6）。

学完这里，你应该能够回答

1. 嵌套结构体怎样逐层访问成员？为什么被嵌套的类型要先定义？
2. `struct Node { int v; Node next; };` 为什么非法？改成什么就合法？
3. 值传递与地址传递在“能否改原数据”和“是否复制”上各有什么不同？
4. 数组作参数时，为什么要多传 len？
5. `const Student *p` 保护了什么？`Student * const p` 保护了什么？

到这里，我们已经解决了什么？ 会嵌套结构体、会用值/地址两种方式传递结构体、会用 const 把“只读”的意图交给编译器检查。**还没有解决的问题：**这些零件怎样拼成一个完整的程序？下一章用笔记里的两个案例——“老师带学生”和“英雄冒泡排序”——把它们全部串起来。

第 7 章　综合案例：把零件拼成程序

前三章学的零件是分散的：定义、数组、指针、嵌套、函数参数。这一章用笔记里的两个案例，把它们拼成完整的程序。看每一步时请问自己：**这一步用到了前面哪个知识点？**

7.1 案例一：老师带毕设学生

> **题意（笔记第 425 页）** 学校正在做毕设，每名老师带 5 个学生，总共 3 名老师。设计学生和老师的结构体：老师的结构体中，有老师姓名和一个存放 5 名学生的数组作为成员；学生的成员有姓名、考试分数。创造数组存放三位老师，通过函数给每个老师所带的学生赋值。

第一步：设计数据。题目里有两种“对象”，所以要两个结构体，而且老师里含有一个学生数组——这是**嵌套 + 数组**的组合，必须**先定义学生，再定义老师**：

数据设计

```
Student: string name; int score;
Teacher: string name; Student arr[5];          注意 arr 是 5 个元素的结构体数组
```

第二步：赋值函数与打印函数。两个函数的形参都是 `Teacher array[]` 加 `int len`（数组不能“自带长度”，见第 6 章）；外层循环遍历老师，内层循环遍历这位老师的 5 名学生。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    int score;
};

struct Teacher
{
    string name;
    Student arr[5];
};

void allocateSpace(Teacher array[], int len)
{
    string nameSeed = "ABCDE";
    for (int i = 0; i < len; i++)
    {
        array[i].name = "Teacher";
        array[i].name += nameSeed[i];
        for (int j = 0; j < 5; j++)
        {
            array[i].arr[j].name = "Student";
            array[i].arr[j].name += nameSeed[j];
            array[i].arr[j].score = 60 + 10 * i + 2 * j;   // 题目要求的“分数”成员
        }
    }
}

void printInfo(Teacher array[], int len)
{
    for (int i = 0; i < len; i++)
    {
```

```
        cout << "老师姓名:" << array[i].name << endl;
        for (int j = 0; j < 5; j++)
        {
            cout << "\t学生姓名:" << array[i].arr[j].name
                 << " 分数:" << array[i].arr[j].score << endl;
        }
    }
}

int main()
{
    Teacher t[3];
    int len = sizeof(t) / sizeof(t[0]);
    allocateSpace(t, len);
    printInfo(t, len);
    return 0;
}
```

```
老师姓名:TeacherA
        学生姓名:StudentA 分数:60
        学生姓名:StudentB 分数:62
        学生姓名:StudentC 分数:64
        学生姓名:StudentD 分数:66
        学生姓名:StudentE 分数:68
老师姓名:TeacherB
        学生姓名:StudentA 分数:70
        ……（共 3 位老师，每位 5 名学生，TeacherC 的分数为 80,82,84,86,88）
```

t[3]：3 位老师；每位老师内含 arr[5]：5 名学生（格内：学生名 / 分数）

t[0] TeacherA	StudentA 60	StudentB 62	StudentC 64	StudentD 66	StudentE 68
t[1] TeacherB	StudentA 70	StudentB 72	StudentC 74	StudentD 76	StudentE 78
t[2] TeacherC	StudentA 80	StudentB 82	StudentC 84	StudentD 86	StudentE 88

arr[0] …arr[4]：这位老师的 5 名学生

分数公式：60 + 10*i + 2*j（i 是老师下标，j 是学生下标）

图 17 案例一的数据在内存中的样子：一个 3×5 的“老师—学生”结构

为什么 allocateSpace 能改 main 里的数组？ 函数形参 Teacher array[] 实际上是指向首元素的**指针**（第 6 章），传进去的是地址——这就是一种**地址传递**。所以函数里 array[i].name = ... 改的是 main 里的 t[i] 本身，不是副本。这也解释了为什么赋值函数**不需要返回值**。

代码片段	用到的知识点
`Student arr[5];`（在 Teacher 里）	嵌套结构体 + 结构体数组
`array[i].arr[j].name`	先数组下标选老师 i，再 .arr，再下标选学生 j，再 .name（多层访问）
`array[i].name += nameSeed[i];`	string 的 += 可以追加一个字符："Teacher" 变 "TeacherA"
`sizeof(t)/sizeof(t[0])`	用数组总字节数除以单个元素字节数求个数
`allocateSpace(t, len)`	数组作参数 = 地址传递，函数可直接改原数组

易错 (1) 内层循环 `j < 5` 的 5 要与 `arr[5]` 一致，一改俱改。(2) 嵌套两层循环时，下标 i、j 不要写反：`array[i].arr[j]` 里 i 选老师、j 选学生。(3) `nameSeed[i]` 的下标不能超过字符串长度（这里 "ABCDE" 长度 5，老师只有 3 位，不会越界）。(4) 笔记里的 score 没有被赋值（纠错 7），读未赋值的成员会得到不确定值。

7.2 案例二：英雄冒泡排序

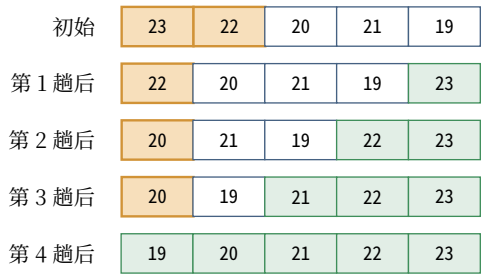
题意（笔记第 427 页） 设计一个英雄的结构体，包括成员姓名、年龄、性别；创建结构体数组，在数组中存放 5 名英雄。通过冒泡排序的算法，将数组中的英雄按照年龄进行**升序**排序，最终打印排序后的结果。数据：刘备 23 男、关羽 22 男、张飞 20 男、赵云 21 男、貂蝉 19 女。

第一步：先弄懂冒泡排序本身。

排序要解决什么？ 有一排数据，想让它从小到大排好。最朴素的想法：**反复比较相邻的两个，如果“前大后小”就交换**。这样每扫一遍，最大的那个就像水里的气泡一样“冒”到最后。重复这个过程，每一遍都会多一个元素就位，所以叫冒泡排序。

冒泡排序（升序）的循环结构

for (i = 0; i < len-1; i++)　**共 $n-1$ 趟**
for (j = 0; j < len-i-1; j++)　**第 i 趟比较前 $n-i$ 个元素**
if (Array[j].age > Array[j+1].age) 交换 Array[j] 与 Array[j+1];



绿色：已经“冒”到最终位置的元素；**橙色**：初始行中第一次被比较的相邻两个（23 与 22）。

每趟结束，右边多一个就位元素，所以需要的趟数是 $n-1=4$；每趟比较范围逐渐缩小，因此内层条件是 `j < len-i-1`。

图 18　升序冒泡排序逐趟结果（年龄序列；每一行都是上一行经过一趟比较交换的结果）

为什么这样写一定能排好？（循环不变量证明） **要证明：**执行完第 i 趟（$i=0,1,\ldots,n-2$）后，数组最后 $i+1$ 个位置上放的是整个数组中最大的 $i+1$ 个数，并且已经按升序排好。
思路：对 i 用数学归纳法。
第 0 趟： j 从 0 扫到 $n-2$，每一次比较都保证 Array[j] 与 Array[j+1] 中的**较大者**留在 $j+1$

> 处。于是“当前扫到的最大值”一路被带着往右走，最终到达最后一个位置 $n-1$。所以最后 1 个位置放的是全体最大值。
> **归纳步：**假设前 i 趟结束后，最后 i 个位置已是最大的 i 个数且有序。第 i 趟只在前 $n-i$ 个位置内（$j < n-i-1$）做同样的扫描，同理把这 $n-i$ 个数的最大者放到位置 $n-i-1$。这个数不大于它后面已就位的元素（因为后面是全体里更大的那批），所以最后 $i+1$ 个位置是最大的 $i+1$ 个数且有序。
> **结论：**做完第 $n-2$ 趟，最后 $n-1$ 个位置已就位，剩下的第一个元素只能是最小的，整个数组有序。证毕。

第二步：为什么交换需要“临时变量”？

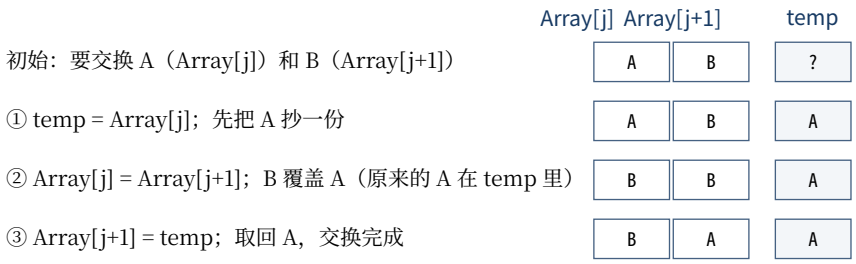


图 19 三步交换：没有 temp，第 ② 步之后 A 就丢了

这里的 `Hero temp = Array[j];` 复制的是整个结构体（姓名、年龄、性别一起搬动），这正是第 4 章“整体赋值＝逐成员拷贝”的应用。所以排序时**整个英雄**一起移动，不会出现“年龄换了、名字没换”的错位。

第三步：完整程序（已纠错并验证）。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Hero
{
    string name;
    int age;
    string sex;
};

void BubbleSort(Hero Array[], int len)
{
    for (int i = 0; i < len - 1; i++)            // 共需 len-1 趟
    {
        for (int j = 0; j < len - i - 1; j++)    // 每趟比较范围逐趟缩小
        {
            if (Array[j].age > Array[j + 1].age)  // 前大后小则交换 => 升序
            {
                Hero temp = Array[j];
                Array[j] = Array[j + 1];
                Array[j + 1] = temp;
            }
        }
    }
}

```

```cpp
void PrintInfo(Hero Array[], int len)
{
    for (int i = 0; i < len; i++)
    {
        cout << "姓名:" << Array[i].name << " 年龄:" << Array[i].age
             << " 性别:" << Array[i].sex << endl;
    }
}

int main()
{
    Hero Array[] =
    {
        {"刘备", 23, "男"},
        {"关羽", 22, "男"},
        {"张飞", 20, "男"},
        {"赵云", 21, "男"},
        {"貂蝉", 19, "女"}
    };
    int len = sizeof(Array) / sizeof(Array[0]);
    BubbleSort(Array, len);
    PrintInfo(Array, len);
    return 0;
}
```

```
姓名:貂蝉 年龄:19 性别:女
姓名:张飞 年龄:20 性别:男
姓名:赵云 年龄:21 性别:男
姓名:关羽 年龄:22 性别:男
姓名:刘备 年龄:23 性别:男
```

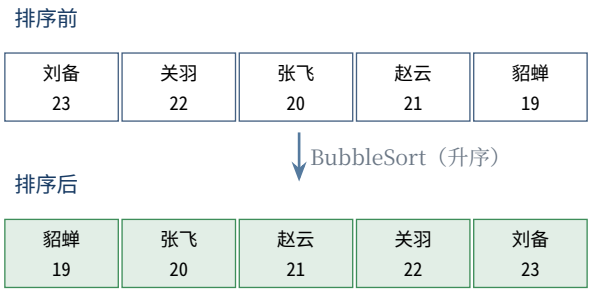


图 20 排序前后：每个“英雄”整体移动，姓名、年龄、性别始终在一起

对照：笔记里的写法会得到什么？ 把比较条件写成 `Array[j+1].age > Array[j].age`（笔记原文），实际运行输出是：刘备23 关羽22 赵云21 张飞20 貂蝉19——**降序**。题目要升序，所以必须把条件换成 `Array[j].age > Array[j+1].age`。（该写法已用 g++ 实测，见 `code/s11_desc.cpp`。）题面最后列出的“结果”实为初始顺序，不是排序结果，参见第 3 章纠错 8(d)。

教学补全：用逐趟追踪验证，以及比较次数

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a[5] = {23, 22, 20, 21, 19};      // 英雄的年龄
    int len = 5, cmp = 0, swp = 0;
    for (int i = 0; i < len - 1; i++)
    {
        for (int j = 0; j < len - i - 1; j++)
        {
            cmp++;
            if (a[j] > a[j + 1])
            {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swp++;
            }
        }
        cout << "第" << i + 1 << "趟后: ";
        for (int k = 0; k < len; k++) cout << a[k] << " ";
        cout << endl;
    }
    cout << "比较 " << cmp << " 次，交换 " << swp << " 次" << endl;
    return 0;
}
```

```
第1趟后: 22 20 21 19 23
第2趟后: 20 21 19 22 23
第3趟后: 20 19 21 22 23
第4趟后: 19 20 21 22 23
比较 10 次，交换 9 次
```

比较次数：第 i 趟比较 $n-1-i$ 次，总共

$$\sum_{i=0}^{n-2}(n-1-i) = (n-1)+(n-2)+\cdots+1 = \frac{n(n-1)}{2}.$$

$n=5$ 时是 10 次，与输出一致；**交换次数**等于“逆序对”的个数（这里是 9 对，如 (23,22)、(23,20) 等）。

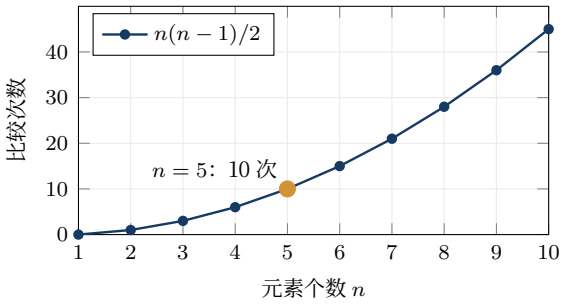


图 21 冒泡排序的比较次数随 n 增长，近似与 n^2 成正比（所以数据量很大时需要更快的排序算法）

知识联系：排序为什么在《数据结构与算法》里一再出现 冒泡排序的外层结构（$n-1$ 趟、范围逐趟缩小）、**以结构体的某一个字段（年龄）作为排序关键字**、整个结构体一起交换，这三件事在后面的选择排序、插入排序、快速排序里都会重复出现；比较次数 $\frac{n(n-1)}{2}$ 就是第一个典型的“时间复杂度 $O(n^2)$”。

学完这里，你应该能够回答

1. 案例一里为什么要先定义 `Student` 再定义 `Teacher`？`allocateSpace` 为什么能修改 `main` 里的数组？
2. `array[i].arr[j].name` 每一段依次是什么？
3. 冒泡排序为什么需要 $n-1$ 趟？为什么内层是 `j < len-i-1`？
4. 交换两个结构体为什么需要 `temp`？它复制了什么？
5. 把比较号改为 `Array[j+1].age > Array[j].age` 会怎样？

到这里，我们已经解决了什么？ 把定义、数组、指针、嵌套、函数参数拼成了两个完整程序，并且知道了冒泡排序为什么对、怎样验证、需要多少次比较。**下一步：**把整讲的易错点、相近概念和知识地图收拢一遍，然后进入 38 道练习——先独立完成，再对答案。

第 8 章　总结：易错点、概念辨析与知识地图

前面七章，每个知识点都是“一步一步长出来”的。这一章把它们收拢：先列最常见的错误，再对比容易混淆的概念，最后画出整张知识地图。

8.1 高频易错点总表

#	错误写法	后果（g++ 实测或行为）	正确写法
1	`struct S {...}` 缺末尾分号	`expected ';' after struct definition`	`struct S {...};`
2	`String name;`	`'String' does not name a type`	`string name;` + `#include <string>`
3	类型名大小写不一致（`student`/`Student`）	找不到类型 / 不完整类型	全篇统一
4	`Student s = {17,"Ala",182};`	类型不匹配，无法转换	按成员声明顺序初始化
5	`int` 成员 `{3.5}`	`narrowing conversion` 报错	用整数，或先显式转换
6	`arr[3]`（只有 3 个元素）	越界，行为未定义	下标 `0..2`，循环 `i < len`
7	对指针用 `.`：`p.age` / `*p.age`	`request for member ... pointer type`	`p->age` 或 `(*p).age`
8	对变量用 `->`	编译报错	`s.age`
9	`Teacher` 先于 `Student` 定义（嵌套）	`field has incomplete type`	先定义内层类型
10	`struct Bad { Bad next; };`	无限大，`incomplete type`	用指针 `Bad *next;`
11	在 `const Student *` 函数里改成员	`assignment of member in read-only object`	去掉修改，或去掉 `const`
12	值传递后期望改动原结构体	只改了副本，原数据不变	用指针或引用
13	函数里对数组形参用 `sizeof(arr)/sizeof(arr[0])`	得到 1（或错误个数）	把 `len` 作参数传入
14	排序比较号方向写反	升序变降序	升序用 `Array[j] > Array[j+1]` 交换
15	交换时不用 `temp`	一个元素被覆盖丢失	`temp = a; a = b; b = temp;`

8.2 相近概念总对比

比较维度	概念 A	概念 B
类型 vs 变量	类型 `Student`：只是图纸，**不占内存**	变量 `s1`：按图纸创建的实物，**占内存**
数组 vs 结构体	同类型元素、靠**下标**访问、个数固定	可不同类型成员、靠**成员名**访问
`.` vs `->`	左边是结构体**变量**	左边是结构体**指针**；`p->m` ≡ `(*p).m`
值传递 vs 地址传递	复制整个结构体；改副本；**原数据安全**	只传地址；可改原数据；不复制，**省时省空间**
地址传递 vs 引用传递	形参是指针，函数内用 `->`，调用写 `&s`	形参是 `Student &`，函数内用 `.`，调用写 `s`
`const` 的位置	`const Student *p`：内容只读，`p` 可改指向	`Student * const p`：指向固定，内容可改
嵌套 vs 指针成员	成员是结构体本身：**必须先完整定义**，占其全部大小	成员是指针：只占一个地址，可指向同类型（链表）
`struct` vs `class`（预告）	`struct`：成员默认公开（`public`）	`class`：成员默认私有（`private`），其余几乎相同

8.3 基础解题步骤总结

遇到结构体题，按这 6 步检查 ① **先看类型：**结构体是怎么定义的？成员的**类型与顺序**是什么？（初始化、输出都依赖这个顺序）
② **再看变量：**每个变量是谁（变量？数组？指针？副本？）。
③ **选访问符：**变量/数组元素用 .，指针用 ->。
④ **看传递方式：**值传递（副本）还是地址/引用（原件）？函数里改动能否回到 main？
⑤ **看 const：**有 const 就不能通过它改成员。
⑥ **画状态：**不确定时，把变量画成“字段—值”的格子，一步一步更新。

8.4 本讲知识地图

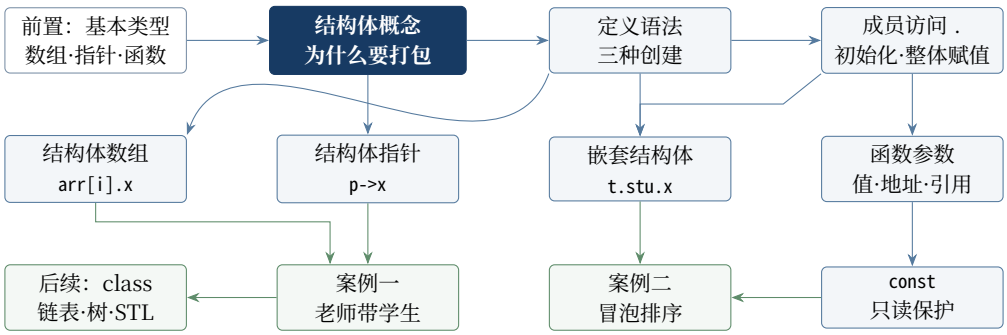


图 22 本讲知识地图（蓝色：核心语法；绿色：综合与后续；箭头表示真实的“依赖”关系）

8.5 后续课程连接

这些知识以后在哪里再次出现？ **类 class：**把结构体的成员再加上**函数成员**，就是类；封装、构造函数、运算符重载都建立在“打包”之上。**链表 / 树 / 图：**每个结点就是带指针成员的结构体，p->next 是遍历的基本动作。**排序与查找：**以结构体的某个字段为关键字，对结构体数组排序、二分查找。**STL：**vector<Student> 保存结构体，sort 配合比较函数按字段排序，pair 是最简单的“二元结构体”。**文件与通信：**记录、数据包头常用结构体描述，要注意对齐。

到这里，我们已经解决了什么？ 整讲的知识已经收拢完毕。**下一步：**用 38 道题检验——25 道基础巩固（单选 10、多选 5、填空 5、解答 5），10 道中等偏下提升（单选 3、多选 2、填空 3、综合大题 2），3 道跨学科迁移（单选 1、多选 1、填空 1）。请先独立完成，再翻到答案。

第 9 章　练习题

本章共 **38 题**：基础巩固 25 题、提升 10 题、跨学科 3 题。答案与解析统一放在题目之后（第 10、11 章）。

9.1 知识点—题目对应表

编号	知识点	对应题号
K1	为什么需要结构体	1
K2	定义语法、类型与变量	2, 3, 11, 16, 21
K3	初始化（顺序、补零、整体赋值）	4, 12, 21
K4	结构体数组与 `len` 公式	5, 13, 17, 22, 34
K5	结构体指针与 `->`	6, 7, 14, 18, 23, 27, 34
K6	嵌套结构体	8, 19, 24, 30, 35
K7	函数传参（值、地址、数组退化）	9, 15, 20, 23, 26, 27, 33, 34, 37
K8	`const`	10, 15, 29, 33, 37, 38
K9	冒泡排序	25, 28, 32, 35
K10	`sizeof`、对齐与填充	17, 26, 31

9.2 A　基础巩固题（25 题）

> **做题提示** 每题先独立作答，再对照答案。凡是配有图的题，请先看图、写出自己的判断，**再**翻到解析区看对应的图解。除另有说明外，所有程序都在 64 位平台、`g++ -std=c++17` 下实际编译运行过。

一、单选题（第 1—10 题）

第 1 题 基础 · 单选　考查：K1 为什么需要结构体

关于“为什么要有结构体”，下列说法最恰当的是：

A. 为了让程序运行得比数组更快
B. 把描述同一个事物、类型却不同的多项数据打包成一个整体，并由我们自己定义新的类型
C. 结构体只是 `int` 的另一个名字
D. 结构体只能保存数字，不能保存字符串

第 2 题　基础 · 单选　考查：K2 结构体的定义语法

下列结构体定义**完全正确**的是：

A. `struct Stu { string name; int age; }`

B. `struct Stu ( string name; int age; );`

C. `struct Stu { string name; int age; };`

D. `struct Stu { string name, int age };`

第 3 题　基础 · 单选　考查：K2 定义类型与创建变量的区别

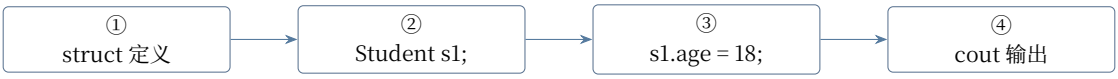
阅读下面的程序。执行到哪一行时，内存里才**第一次出现**名为 `s1` 的那个学生？

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; };   // ①

int main()
{
    Student s1;                             // ②
    s1.age = 18;                            // ③
    cout << s1.age << endl;                 // ④
    return 0;
}
```

【题目区图：程序的执行顺序】



程序按 ①→②→③→④ 依次执行；问题：从哪一步起，内存里有了一个叫 s1 的学生？

A. ①

B. ②

C. ③

D. ④

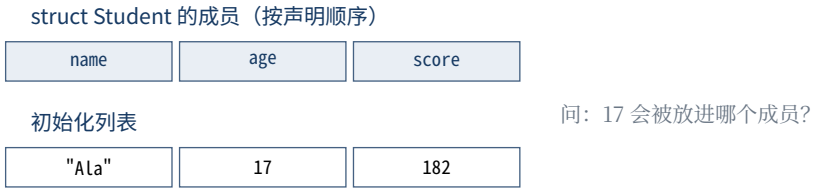
第 4 题　基础 · 单选　考查：K3 按顺序初始化

已知结构体和变量如下，则 `s2.age` 的值是：

```cpp
struct Student
{
    string name;
    int age;
    int score;
};

Student s2 = {"Ala", 17, 182};
```

【题目区图：成员的声明顺序与初始化列表】

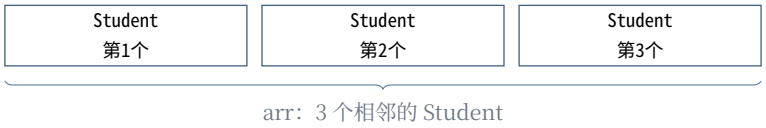


A. 182
B. 17
C. "Ala"
D. 编译错误

第 5 题　基础 · 单选　考查：K4 结构体数组与下标

已定义 Student arr[3];，要访问**第 2 个**同学的年龄，应写：

【题目区图：3 个相邻的元素】



A. arr[2].age
B. arr[1].age
C. arr(1).age
D. arr.age[1]

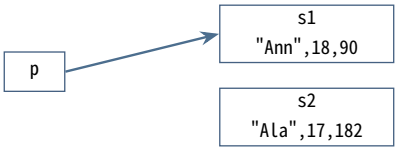
第 6 题　基础 · 单选　考查：K5 结构体指针的含义

执行下面的语句后，哪一幅图正确表示了 p 与 s1、s2 的关系？

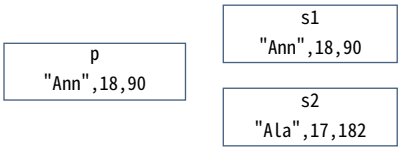
```
Student s1 = {"Ann", 18, 90};
Student *p = &s1;
```

【题目区图（选项）：四种可能的关系，图中比例一致】

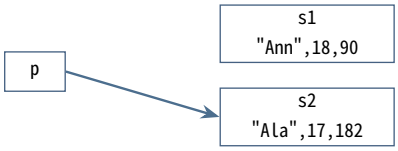
A.



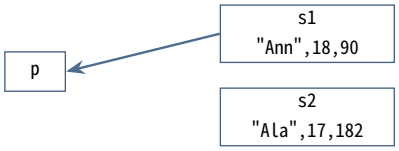
B.



C.



D.



第 7 题　基础 · 单选　考查：K5 箭头运算符

已知 `Student *p = &s1;`，下列与 `p->age` 完全等价的是：

A. `*p.age`

B. `(*p).age`

C. `&p.age`

D. `p.age`

第 8 题　基础 · 单选　考查：K6 嵌套结构体的成员访问

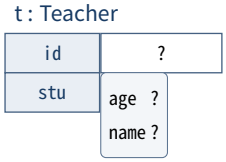
已知下面的定义，要把老师 t 所带学生的年龄设为 17，应写：

```
struct Student
{
    int age;
    string name;
};

struct Teacher
{
    int id;
    Student stu;
};

Teacher t;
```

【题目区图：Teacher 里面套着 Student】



stu 本身又是一个完整的 Student

A. `t.age = 17;`

B. `t.stu.age = 17;`

C. `t.stu->age = 17;`

D. `t.Student.age = 17;`

第 9 题 基础 · 单选 考查：K7 值传递

下面程序的输出是：

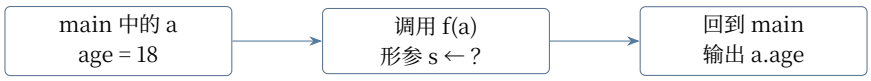
```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; };

void f(Student s)
{
    s.age = 99;
}

int main()
{
    Student a = {"A", 18};
    f(a);
    cout << a.age << endl;
    return 0;
}
```

【题目区图：调用前后涉及的变量】



A. 99
B. 18
C. 编译错误
D. 结果不确定

第 10 题 基础 · 单选 考查：K8 const 修饰结构体指针

已知 `Student s = {"A", 18}; const Student *p = &s;`，下列说法正确的是：

A. 以后 s 本身也不能再被修改
B. 不能通过 p 修改 s 的成员（如 `p->age = 20;` 会报错）
C. p 不能再指向别的 Student 变量
D. p 必须初始化为 `nullptr`

二、多选题（第 11—15 题；每题有两个或两个以上正确选项）

第 11 题　基础 · 多选　考查：K2 结构体基础说法

关于结构体，下列说法正确的有：

A. 一个结构体可以把不同类型的成员（如 `string`、`int`）放在一起

B. 结构体定义结束的右花括号后面必须写分号

C. 写下 `struct Student {…};` 的同时，编译器就为一个学生分配了内存

D. 创建了结构体变量 `s` 之后，用 `s.age` 访问它的成员

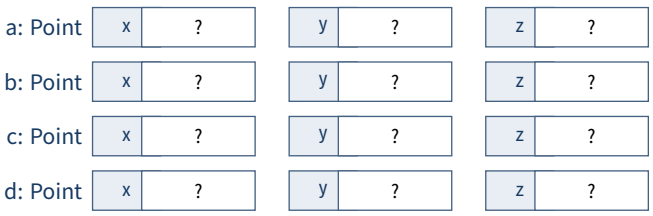
第 12 题　基础 · 多选　考查：K3 初始化：补零、整体赋值

阅读下面的代码（均在 `main` 函数内，编译通过）。下列说法正确的有：

```
struct Point { int x; int y; int z; };

Point a = {1, 2};
Point b = {};
Point c = {3, 4, 5};
Point d = c;
d.x = 99;
```

【题目区图：四个变量刚被创建、尚未看结果】



A. `a.z` 的值是 0

B. `b.x`、`b.y`、`b.z` 都是 0

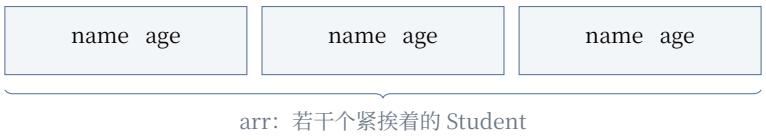
C. 执行 `d.x = 99;` 之后，`c.x` 也变成 99

D. `d.y` 的值是 4

第 13 题　基础 · 多选　考查：K4 结构体数组

已知 `struct Student { string name; int age; };`，关于结构体数组，下列说法正确的有：

【题目区图：数组里的每个元素都是一个完整的 Student】



A. `Student arr[3];` 有 3 个元素，合法下标是 0、1、2

B. `Student arr[] = {{"A",1},{"B",2}};` 的元素个数是 2

C. 若 `arr` 是真正的数组名，则 `sizeof(arr)/sizeof(arr[0])` 等于元素个数

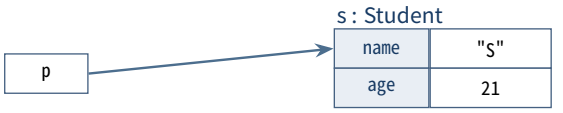
D. `Student arr[3] = {{"A",1},{"B",2},{"C",3},{"D",4}};` 可以通过编译

第 14 题　基础 · 多选　考查：K5 指针访问成员

阅读下面的程序，下列说法正确的有：

```cpp
#include <iostream>
#include <string>
using namespace std;
struct Student { string name; int age; };
int main()
{
    Student s = {"S", 21};
    Student *p = &s;
    cout << p->age << " " << (*p).age << " " << (&s)->age << " " << s.age << endl;
    return 0;
}
```

【题目区图：p 与 s 的关系】



四种写法各会得到什么？

A. 程序输出四个相同的数 21 21 21 21
B. p 里保存的是 s 的地址
C. *p.age 与 (*p).age 等价
D. (&s)->age 是合法写法

第 15 题　基础 · 多选　考查：K7、K8 三种传参与 const

下列关于结构体作函数参数的说法，正确的有：

【题目区图：三种调用方式，形参各拿到了什么？】

调用：f(a) void f(Student s)	调用：f(&a) void f(Student *p)	调用：f(a) void f(const Student *p)
形参拿到：？	形参拿到：？	形参拿到：？

A. 值传递时，函数里修改形参，不会改变实参
B. 地址传递时，函数里通过指针修改成员，会改变原结构体
C. 形参写成 const Student *p 时，不能通过 p 修改成员
D. 值传递时，不会产生副本，形参就是实参本身

三、填空题（第 16—20 题）

第 16 题 基础 · 填空　考查：K2 定义结构体的关键字与分号

定义结构体要使用关键字 _____，右花括号后面必须有 _____。

第 17 题 基础 · 填空　考查：K4、K10 数组大小与元素个数（设 int 为 4 字节，成员间无填充）

```
struct P { int x; int y; };   P a[] = {{1,2},{3,4},{5,6},{7,8},{9,10}};
```

则 sizeof(P) = _____，sizeof(a) = _____，len = sizeof(a)/sizeof(a[0]) = _____。

【题目区图：一个 P 的内部与初始化数据】



第 18 题 基础 · 填空　考查：K5 通过指针修改成员

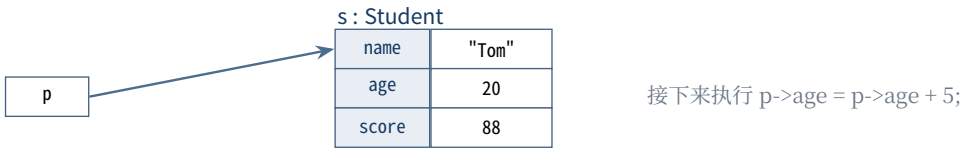
下面程序输出 _____。

```
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; int score; };

int main()
{
    Student s = {"Tom", 20, 88};
    Student *p = &s;
    p->age = p->age + 5;
    cout << s.age << endl;
    return 0;
}
```

【题目区图：执行前的状态】



第 19 题 基础 · 填空　考查：K6 嵌套结构体

下面程序输出 _____。

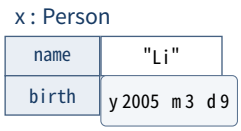
```
#include <iostream>
#include <string>
using namespace std;

struct Date { int y; int m; int d; };
struct Person { string name; Date birth; };

int main()
{
```

```
    Person x = {"Li", {2005, 3, 9}};
    cout << x.birth.m << endl;
    return 0;
}
```

【题目区图：Person 里嵌着 Date】



birth 本身是一个 Date

第 20 题 基础 · 填空 考查：K7 地址传递与值传递混合

下面程序输出 _____。

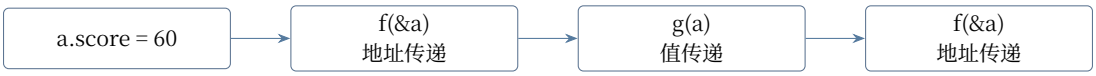
```
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; int score; };

void f(Student *p) { p->score += 10; }
void g(Student s)  { s.score += 100; }

int main()
{
    Student a = {"A", 18, 60};
    f(&a);
    g(a);
    f(&a);
    cout << a.score << endl;
    return 0;
}
```

【题目区图：三次调用的先后】



四、解答题（第 21—25 题）

第 21 题 基础 · 解答 考查：K2、K3 定义、创建并输出

定义结构体 Book，成员为书名 title（string）、价格 price（double）、页数 pages（int）。在 main 中用初始化列表创建 b1（书名“C++入门”，价格 59.5，页数 420），并按**书名、价格、页数**的顺序输出，用空格隔开。写出完整程序。

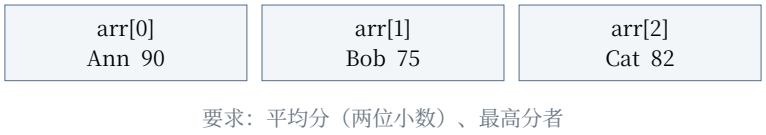
【题目区图：目标——把数据放进模板，再依次取出】



第 22 题 基础 · 解答 考查：K4 结构体数组、求和与最大值

已知 struct Student { string name; int score; };，用数组保存 3 名学生：Ann 90 分、Bob 75 分、Cat 82 分。用 sizeof 公式求元素个数，输出**平均分（保留两位小数）**以及**最高分同学的姓名和分数**。写出完整程序并给出运行结果。

【题目区图：数据】



要求：平均分（两位小数）、最高分者

第 23 题 基础 · 解答 考查：K5、K7 用指针修改原结构体

Student s = {"Dan", 70};（成员 name、score）。(1) 写函数 void addScore(Student *p, int d)，使调用 addScore(&s, 15) 后 s.score 变成 85；(2) 若把形参改成 Student p（值传递），调用后 s.score 是多少？为什么？

【题目区图：两种调用，原来的 score 都是 70】



第 24 题 基础 · 解答 考查：K6 嵌套结构体的定义顺序

下面程序编译不通过。(1) 指出错误的原因；(2) 改正程序；(3) 写出改正后的输出。

```
#include <iostream>
#include <string>
using namespace std;

struct Teacher
{
    string name;
    Student stu;
};

```

```cpp
struct Student
{
    string name;
    int score;
};

int main()
{
    Teacher t;
    t.name = "Zhao";
    t.stu.name = "Sun";
    t.stu.score = 95;
    cout << t.name << ":" << t.stu.name << " " << t.stu.score << endl;
    return 0;
}
```

【题目区图：Teacher 的定义里用到了 Student】



编译器从上往下读：读到 Teacher 时，认识 Student 吗？

第 25 题　基础 · 解答　考查：K9 冒泡排序的手工模拟

对数组 {5, 3, 4, 1, 2} 做**升序**冒泡排序（外层 i 从 0 到 len-2，内层 j 从 0 到 len-i-2，相邻逆序就交换）。写出**每一趟结束后**数组的内容，并求总比较次数与总交换次数。

【题目区图：初始数组】



9.3　B　提升题（10 题）

> **提升题说明**　提升题把前面的知识组合起来：要先在脑子里（或纸上）画出内存图，再动笔。第 34、35 题是综合大题，建议先画图再写。

一、单选题（第 26—28 题）

第 26 题　提升 · 单选　考查：K7、K10 数组形参退化（设 64 位平台：int 4 字节、指针 8 字节）

下面程序的输出是：

```cpp
#include <iostream>
using namespace std;

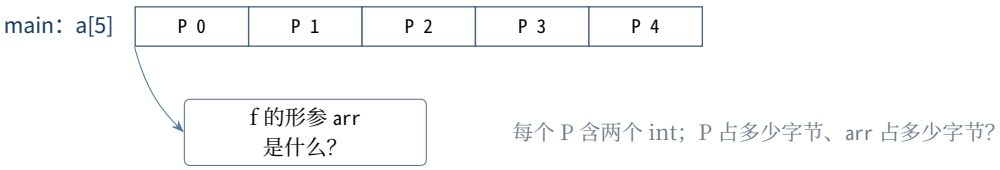
struct P { int x, y; };

void f(P arr[])
{
    cout << sizeof(arr) / sizeof(arr[0]);
}

int main()
```

```
{
    P a[5];
    cout << sizeof(a) / sizeof(a[0]) << " ";
    f(a);
    cout << endl;
    return 0;
}
```

【题目区图：main 里的数组与 f 里的形参】



A. 5 5
B. 5 1
C. 5 8
D. 编译错误

第 27 题 提升 · 单选 考查：K5、K7 用指针交换

下面程序的输出是：

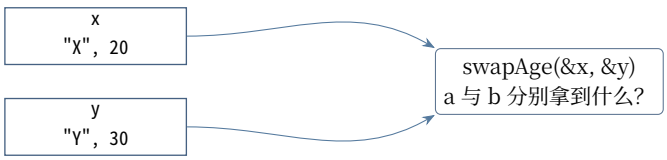
```
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; };

void swapAge(Student *a, Student *b)
{
    int t = a->age;
    a->age = b->age;
    b->age = t;
}

int main()
{
    Student x = {"X", 20}, y = {"Y", 30};
    swapAge(&x, &y);
    cout << x.age << " " << y.age << endl;
    return 0;
}
```

【题目区图：调用前的 x、y 与函数的两个形参】



【题目区图（选项）：四种可能的结果，格式为「x.age　y.age」】

A.

x.age	y.age
20	30

B.

x.age	y.age
30	20

C.

x.age	y.age
30	30

D.

x.age	y.age
20	20

第 28 题　提升 · 单选　考查：K9 冒泡排序的比较方向

下面是笔记中冒泡排序的写法（比较条件是 `Array[j + 1].age > Array[j].age`）。排序完成后，各英雄年龄依次是：

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Hero { string name; int age; string sex; };

void BubbleSortNote(Hero Array[], int len)     // 笔记中的比较方向：后面比前面大才交换
{
    for (int i = 0; i < len - 1; i++)
        for (int j = 0; j < len - i - 1; j++)
            if (Array[j + 1].age > Array[j].age)
            {
                Hero temp = Array[j];
                Array[j] = Array[j + 1];
                Array[j + 1] = temp;
            }
}

int main()
{
    Hero Array[] = {{"刘备", 23, "男"}, {"关羽", 22, "男"}, {"张飞", 20, "男"},
                    {"赵云", 21, "男"}, {"貂蝉", 19, "女"}};
    BubbleSortNote(Array, 5);
    for (int i = 0; i < 5; i++) cout << Array[i].name << Array[i].age << " ";
    cout << endl;
    return 0;
}
```

【题目区图：排序前的年龄，与比较条件】

23	22	20	21	19

← 排序前

交换条件：后一个 **大于** 前一个 `Array[j+1].age > Array[j].age`

【题目区图（选项）：排序完成后的年龄】

A.

19	20	21	22	23

B.

23	22	21	20	19

C.

23	22	20	21	19

D.

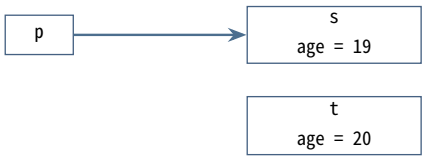
22	20	21	19	23

二、多选题（第 29—30 题）

第 29 题 提升 · 多选　考查：K8 const 的三个位置（Student s, t; 均已定义）

下列语句**单独**放在 main 中，会**编译报错**的有：

【题目区图：p 已经指向 s，t 是另一个学生】



下面的语句想对 p 做什么?

A. `const Student *p = &s; p->age = 20;`

B. `const Student *p = &s; p = &t;`

C. `Student *const p = &s; p = &t;`

D. `const Student *const p = &s; p->age = 20;`

第 30 题 提升 · 多选　考查：K6 嵌套结构体

已知 `struct Student { int age; string name; };` 和 `struct Teacher { int id; Student stu; };`（Student 定义在前），`Teacher t;`。下列说法正确的有：

【题目区图：Teacher 的“零件”】



A. `t.stu.age = 17;` 是合法语句

B. 若把 Teacher 的定义移到 Student 之前，会因 Student 不完整而报错

C. `struct Student { Student s; int a; };`（结构体里套自己）是合法的

D. 若 `Teacher *p = &t;`，则 `p->stu.age = 17;` 是合法语句

三、填空题（第 31—33 题）

第 31 题 提升 · 填空　考查：K10 内存对齐（设 64 位平台：char 1 字节、int 4 字节且按 4 字节对齐）

`struct S1 { char a; int b; char c; };`　`struct S2 { int b; char a; char c; };`

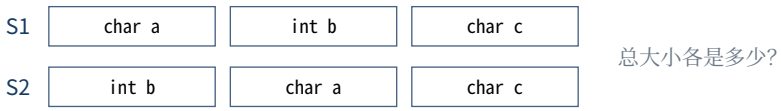
则 `sizeof(S1)` = _____，`sizeof(S2)` = _____。

```cpp
#include <iostream>
using namespace std;

struct S1 { char a; int b; char c; };
struct S2 { int b; char a; char c; };

int main()
{
    cout << sizeof(S1) << " " << sizeof(S2) << endl;
    return 0;
}
```

【题目区图：两个结构体的成员与类型尺寸（成员相同，仅顺序不同）】



第 32 题　提升 · 填空　考查：K9 冒泡排序一趟的结果与比较次数

对数组 {23，22，20，21，19} 做升序冒泡排序（5 个元素）。第 1 趟结束后数组为 _____；整个排序过程共比较 _____ 次。

【题目区图：初始数组与第 1 趟要比较的相邻对】



第 33 题　提升 · 填空　考查：K7、K8 值传递、地址传递与 const

下面程序输出 _____。

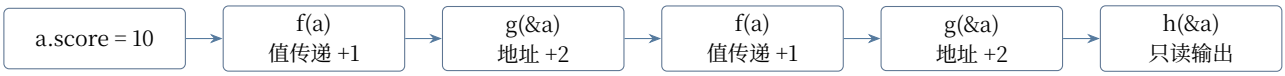
```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int age; int score; };

void f(Student s)        { s.score += 1; }
void g(Student *p)       { p->score += 2; }
void h(const Student *p) { cout << p->score << endl; }

int main()
{
    Student a = {"A", 0, 10};
    f(a);
    g(&a);
    f(a);
    g(&a);
    h(&a);
    return 0;
}
```

【题目区图：a.score 起始为 10，之后依次经历四次调用】



四、综合大题（第 34—35 题）

第 34 题　提升 · 综合　考查：K4、K5、K7 返回结构体指针

阅读下面程序，回答：(1) 写出程序的三行输出；(2) 为什么第一行是 `Bob 96`，而且 `arr[1].score` 也变成了 96？(3) `findMaxCopy` 返回的是副本，为什么 `arr2[1].score` 没变？

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int score; };

Student *findMax(Student a[], int n)      // 返回指向“最高分那个元素”的指针
{
    Student *best = &a[0];
    for (int i = 1; i < n; i++)
        if (a[i].score > best->score)
            best = &a[i];
    return best;
}

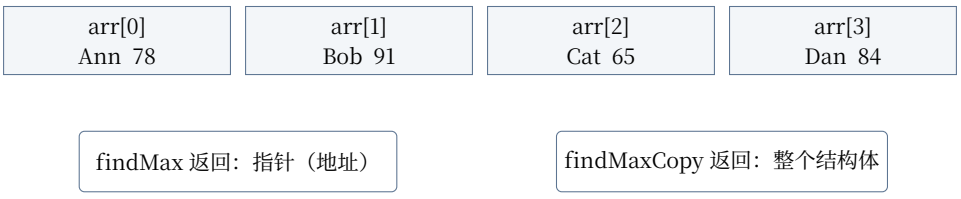
Student findMaxCopy(Student a[], int n)   // 对比：返回副本
{
    int k = 0;
    for (int i = 1; i < n; i++)
        if (a[i].score > a[k].score) k = i;
    return a[k];
}

int main()
{
    Student arr[4] = {{"Ann", 78}, {"Bob", 91}, {"Cat", 65}, {"Dan", 84}};

    Student *p = findMax(arr, 4);
    p->score += 5;
    cout << p->name << " " << p->score << endl;
    cout << "arr[1].score = " << arr[1].score << endl;

    Student arr2[4] = {{"Ann", 78}, {"Bob", 91}, {"Cat", 65}, {"Dan", 84}};
    Student q = findMaxCopy(arr2, 4);
    q.score += 5;
    cout << "arr2[1].score = " << arr2[1].score << ", q.score = " << q.score << endl;
    return 0;
}
```

【题目区图：4 名学生与两个函数的返回方式】



第 35 题　提升 · 综合　考查：K6、K9 结构体数组里的结构体数组

阅读下面程序，回答：(1) 标号 (1) 处输出什么？(2) 标号 (2) 处输出什么（三行）？(3) 标号 (3) 处输出什么？(4)（变式思考）若要找出**某位老师所带学生中的最高分**，你会如何修改？

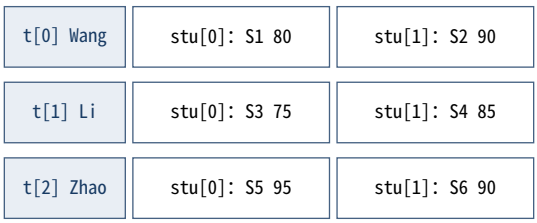
```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int score; };
struct Teacher { string name; Student stu[2]; };

double average(const Teacher *t)
{
    return (t->stu[0].score + t->stu[1].score) / 2.0;
}

int main()
{
    Teacher t[3] =
    {
        {"Wang", {{"S1", 80}, {"S2", 90}}},
        {"Li",   {{"S3", 75}, {"S4", 85}}},
        {"Zhao", {{"S5", 95}, {"S6", 90}}}
    };
    cout << t[1].stu[0].score << endl;            // (1)
    int best = 0;
    for (int i = 0; i < 3; i++)
    {
        cout << t[i].name << " 平均分 " << average(&t[i]) << endl;   // (2)
        if (average(&t[i]) > average(&t[best])) best = i;
    }
    cout << "最高: " << t[best].name << endl;      // (3)
    return 0;
}
```

【题目区图：3 位老师、每位带 2 名学生】



9.4 C　跨学科题（3 题）

> **说明**　跨学科题借用其他学科的**情境**，但所需知识仍是本专题（结构体）的内容。每题的背景都已在题干里交代清楚，**不需要**额外的物理、物流或控制学知识。

第 36 题　跨学科 · 单选　考查：物理背景下的嵌套结构体与成员访问

> **背景（自包含）**　物理里常用“位置 + 速度”描述一个质点：在很短的时间 Δt（秒）内，质点位置的变化量 = 速度 × Δt，即 $x \leftarrow x + v_x \Delta t$，$y \leftarrow y + v_y \Delta t$。本题只需做这两个乘法和加法。

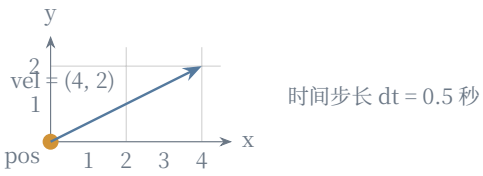
下面程序输出的是哪一个点？

```cpp
#include <iostream>
using namespace std;

struct Vec2 { double x; double y; };
struct Particle { Vec2 pos; Vec2 vel; };

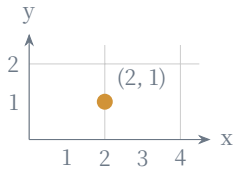
int main()
{
    Particle q = {{0, 0}, {4, 2}};
    double dt = 0.5;
    q.pos.x += q.vel.x * dt;
    q.pos.y += q.vel.y * dt;
    cout << "(" << q.pos.x << ", " << q.pos.y << ")" << endl;
    return 0;
}
```

【题目区图：质点的初始位置与速度（每格 1 个单位）】

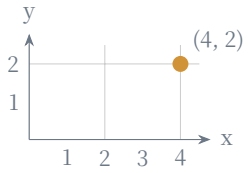


【题目区图（选项）：四个候选点，坐标刻度一致】

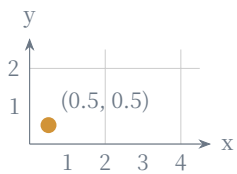
A.



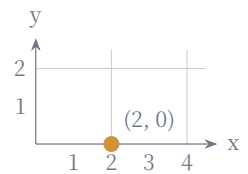
B.



C.



D.



第 37 题　跨学科 · 多选　考查：物流背景下的结构体数组、const 与数组退化

> **背景（自包含）** 仓库清单里每种货物有名称、单件重量（千克）和件数；一种货物的总重量 = 单件重量 × 件数，整张清单的总重量是各种货物总重量之和。

阅读下面程序，下列说法正确的有：

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Item { string name; double weight; int qty; };

double total(const Item *a, int n)
{
    double sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i].weight * a[i].qty;
    return sum;
}

int main()
{
    Item items[2] = {{"螺丝", 0.5, 10}, {"铁板", 3.0, 2}};
    cout << total(items, 2) << endl;
    return 0;
}
```

【题目区图：清单中的两种货物】

items[0] : Item

name	"螺丝"
weight	0.5
qty	10

items[1] : Item

name	"铁板"
weight	3.0
qty	2

A. 在 main 中，sizeof(items)/sizeof(items[0]) 的值是 2

B. total 的形参写成 const Item *a，表示函数不会修改清单中的数据

C. 在 total 函数里写 sizeof(a)/sizeof(a[0]) 同样能得到 2

D. 程序输出 11

第 38 题　跨学科 · 填空　考查：控制背景下的结构体复制与 const 指针

> **背景（自包含）** 自动控制里常把一组控制参数 K_p, K_i, K_d 放在一起。其中“比例项”的输出 $= K_p \times e$，e 是当前误差。本题只需做这一次乘法。

下面程序第一行输出 _____；第二行输出 _____ _____（两个数，空格隔开）。

```cpp
#include <iostream>
using namespace std;

struct Pid { double kp, ki, kd; };

double pTerm(const Pid *c, double e)
```

```
{
    return c->kp * e;
}

int main()
{
    Pid c = {1.5, 0.2, 0.1};
    cout << pTerm(&c, 2) << endl;
    Pid c2 = c;
    c2.kp = 4;
    cout << c.kp << " " << c2.kp << endl;
    return 0;
}
```

【题目区图：参数结构体 c】

c : Pid

kp	1.5
ki	0.2
kd	0.1

先用 pTerm(&c, 2)；再复制出 c2，改 c2.kp

第 10 章　答案速查

这一章只给**最终答案**，方便你先自己核对；想知道“为什么”，请翻到第 11 章看逐题图解。

基础巩固题

题号	答案	题号	答案
1	B	14	ABD
2	C	15	ABC
3	B（②）	16	`struct`；分号 `;`
4	B（17）	17	8；40；5
5	B	18	25
6	A	19	3
7	B	20	80
8	B	21	见第 11 章（程序与输出）
9	B（18）	22	平均分 82.33；最高分 Ann（90）
10	B	23	(1) 85；(2) 70
11	ABD	24	先定义 `Student`；输出 `Zhao:Sun 95`
12	ABD	25	见第 11 章：比较 10 次，交换 8 次
13	ABC		

提升题

题号	答案	题号	答案
26	B（`5 1`）	31	12；8
27	B（`30 20`）	32	`22 20 21 19 23`；10 次
28	B（`23 22 21 20 19`）	33	14
29	ACD	34	见第 11 章（`Bob 96` 等）
30	ABD	35	见第 11 章（75；85、80、92.5；Zhao）

跨学科题

题号	答案
36	A，即 $(2, 1)$
37	ABD
38	3；`1.5 4`

第 11 章　逐题解析与图解

11.1　基础题解析（第 1—25 题）

> **怎样读解析**　每题先给**结论**，再给**依据**（回到第 4—7 章的哪条规则），带图的题再给**图解**：左边是过程，右边是结果。错误选项会说明“它实际上是什么样子、和哪条规则矛盾”。

单选题解析

第 1 题　基础 · 单选　答案：B

依据：第 4 章“为什么需要结构体”——描述一个学生要同时用姓名（string）、年龄（int）、成绩等不同类型的数据，数组只能放同类型元素，所以要把它们**打包**成一个整体并自己定义新类型。

A 错：结构体不是为了提速。**C 错：**它是用户自定义的**新类型**，不是 int 的别名。**D 错：**成员可以是字符串、数组甚至别的结构体。

第 2 题　基础 · 单选　答案：C

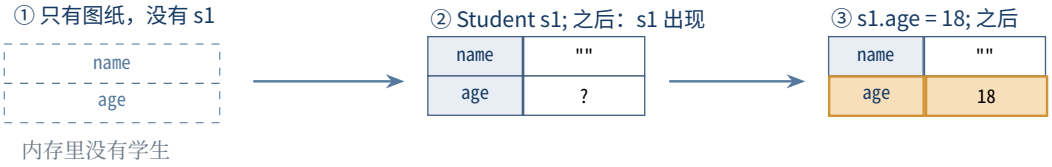
依据：语法模板 `struct 名字 { 成员; };`：花括号、成员后各有分号，右花括号后**还有一个分号**。

A 错：末尾缺分号。**B 错：**用了圆括号。**D 错：**成员之间用逗号隔开了不同类型的声明，每个成员声明必须各自以分号结束。

第 3 题　基础 · 单选　答案：B（②）

依据：`struct Student {…};` 只是**画图纸**（定义类型），不占内存；`Student s1;` 才是“按图纸造房子”（创建变量），此时内存里才出现 s1。

【解析区图：①②③ 三步中内存里发生了什么】

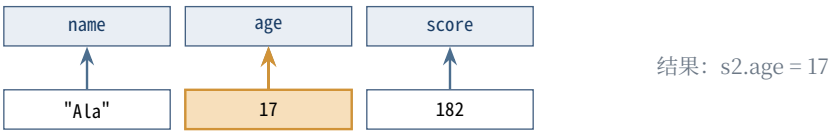


A 错：①是定义类型。**C 错：**③是给已有的 s1 的成员赋值（s1 在②就已存在）。**D 错：**④只是输出。

第 4 题　基础 · 单选　答案：B（17）

依据：大括号初始化按**成员声明顺序**逐个对应：第 1 个值给 name，第 2 个给 age，第 3 个给 score。

【解析区图：初始化列表与成员一一对应】

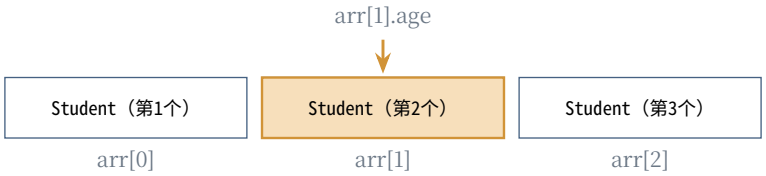


A 错：182 是 score。**C 错：**"Ala" 是 name。**D 错：**类型与个数都匹配，能编译。

第 5 题　基础 · 单选　答案：B

依据：下标从 0 开始，第 2 个元素下标为 1；先用 [1] 选元素，再用 .age 取成员。

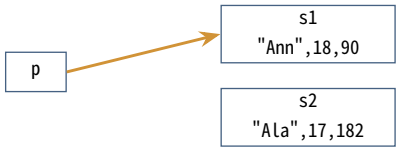
【解析区图：下标从 0 数起】



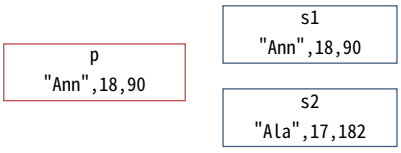
A 错：arr[2] 是第 3 个。**C 错：**取元素用方括号，不是圆括号。**D 错：**arr 是数组，没有叫 age 的成员。

第 6 题　基础 · 单选　答案：A

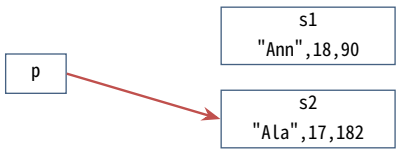
依据：&s1 是 s1 的地址；Student *p = &s1; 让 p 里保存这个地址，所以图上是一根箭头从 p 指向 s1。p 自己很小，只存一个地址。



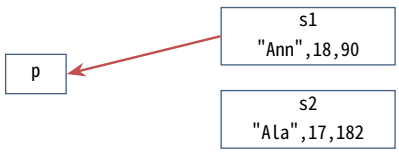
A 正确 p 保存 s1 的地址，箭头指向 s1。符合定义。



B 错误 图中 p 里装了一份学生数据，说明把 p 当成了结构体。实际上 p 只装地址，并**没有**创建新学生（第 5 章“指针不是结构体本身”）。



C 错误 箭头指向了 s2。但语句是 p = &s1，取的是 s1 的地址，不是 s2 的。



D 错误 箭头方向画反了：是 p “记住” s1，不是 s1 记住 p。

第 7 题　基础 · 单选　答案：B

依据：第 5 章核心等价式 p->成员 ⟺ (*p).成员。

A 错：. 优先级高于 *，*p.age 被读成 *(p.age)，而指针没有成员，编译报错。**C 错：**&p.age 同理，先对指针用 .。**D 错：**p 是指针，不能直接用 .，应写 ->。

第 8 题　基础 · 单选　答案：B

依据：嵌套结构体逐层用点号“剥开”：t 的成员 stu，stu 的成员 age。

【解析区图：访问路径 t → stu → age】

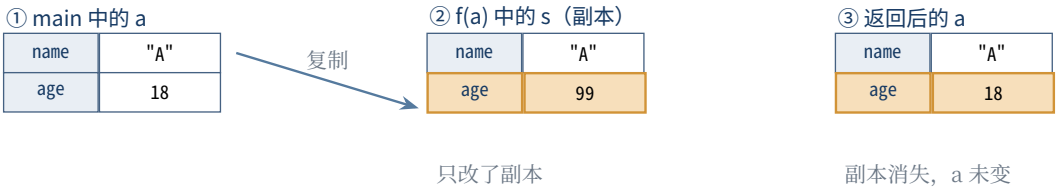


A 错：Teacher 没有 age 成员（成员是 id 与 stu）。**C 错：**t.stu 是 Student 变量，不是指针，不能用 ->。**D 错：**Student 是类型名，不是成员名。以上三条均已用 g++ 验证报错。

第 9 题　基础 · 单选　答案：B（18）

依据：值传递把实参**复制**给形参，函数里改的是副本，原来的 a 不变（第 6 章）。

【解析区图：题意 → 过程 → 结果】



A 错：把副本当成了原件。**C 错：**语法完全正确。**D 错：**结果完全确定，是 18。

第 10 题　基础 · 单选　答案：B

依据：const Student *p 中 const 修饰“*p 指向的内容”：**通过 p 不能改成员。**
A 错：s 自己不是 const，仍可写 s.age = 20;。**C 错：**p 本身可以改指向别人（那是 Student *const p）。**D 错：**初始化为 &s 即可，没有必须为 nullptr 的规定。第 29 题会把三种位置放在一起比较。

多选题解析

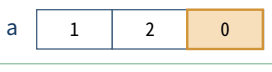
第 11 题　基础 · 多选　答案：ABD

依据：第 4 章定义语法与“类型 vs 变量”。**A、B、D 正确。**
C 错：struct Student {…}; 只是定义类型（画图纸），**不分配内存**；创建变量时才分配（第 3 题）。

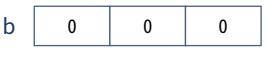
第 12 题　基础 · 多选　答案：ABD

依据：聚合初始化（第 4 章）：给少了的成员补 0；{} 全部补 0；d = c 是整体复制，复制后两个变量互相独立。

【解析区图：逐个选项看四个变量的真实内容（格内为最终值）】



A 正确 Point a = {1, 2} 只给了 x、y，z 被补 0。



B 正确 空花括号 {}：三个成员全部补 0。



C 错误 d = c 之后 d 是 c 的**副本**，改 d.x 不影响 c；c.x 仍是 3。

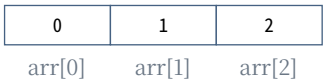


D 正确 d 复制自 c = {3,4,5}，只有 x 被改成 99，d.y 仍是 4。

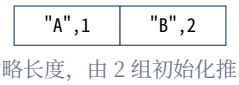
第 13 题　基础 · 多选　答案：ABC

依据：第 5 章结构体数组与 len 公式。

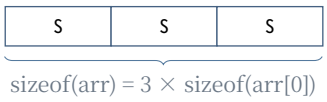
【解析区图：逐个选项】



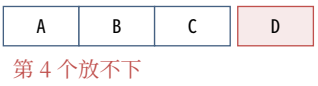
A 正确 3 个元素，合法下标 0、1、2；arr[3] 越界。



B 正确 两组花括号 → 两个元素。



C 正确 整个数组字节数 $=n\times$ 单个元素字节数，两边相除得 $n=3$（前提：arr 是真数组名）。

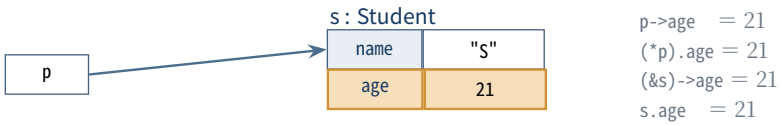


D 错误 初始化项比元素个数多，g++ 报 too many initializers for 'Student [3]'（已验证）。

第 14 题　基础 · 多选　答案：ABD

依据：第 5 章：p->age ⟺ (*p).age；&s 本身也是一个指针。

【解析区图：p 与 s；四种写法殊途同归】



A、B、D 正确。C 错：. 的优先级高于 *，*p.age 被读成 *(p.age)——先对指针 p 取成员，指针没有成员，编译报错（e05 已验证）；必须写成 (*p).age。

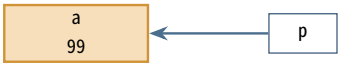
第 15 题　基础 · 多选　答案：ABC

依据：第 6 章三种传参方式与 const。

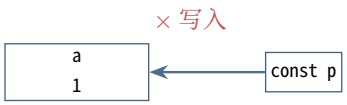
【解析区图：逐个选项】



A 正确 形参是副本；改副本，实参 a 仍为 1。



B 正确 p 存的是 a 的地址，p->成员 改的就是 a 本身。



C 正确 const 锁住“通过 p 写入”，p->age = 20; 会编译报错。



D 错误 值传递会产生副本，形参是新的变量，并非实参本身。

填空题解析

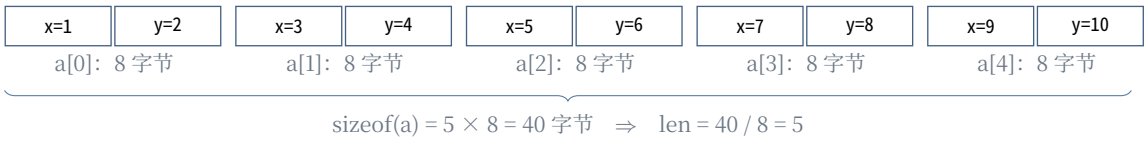
第 16 题　基础 · 填空　答案：struct；分号

依据：关键字 struct；右花括号后的分号（第 2 题 A、B、D 错在缺分号或括号写法）。

第 17 题　基础 · 填空　答案：8；40；5

依据：两个 int 共 $4+4=8$ 字节；数组 a 有 5 组初始化，故 5 个元素，总大小 $5\times 8=40$；len $=40/8=5$。

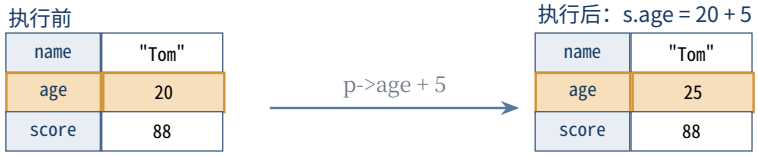
【解析区图：题意 → 过程 → 结果】



第 18 题　基础 · 填空　答案：25

依据：p 指向 s，p->age 就是 s.age。右边先读到 20，加 5 得 25，写回 s.age。

【解析区图：执行前 → 执行后】



第 19 题　基础 · 填空　答案：3

依据：x.birth 是 Date，x.birth.m 是它的月份成员，初始化 {2005，3，9} 按顺序给 y，m，d。

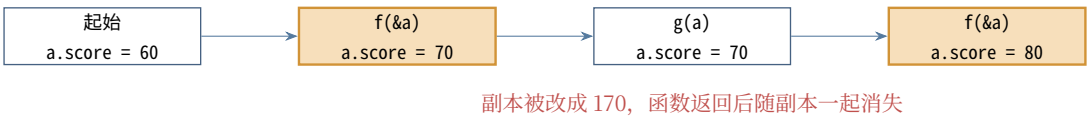
【解析区图：路径 x → birth → m】



第 20 题　基础 · 填空　答案：80

依据：f(&a) 通过地址改原值（+10）；g(a) 改的是副本（+100 只加在副本上，随后丢弃）；再次 f(&a) 又 +10。$60+10+10=80$。

【解析区图：score 随四个步骤的变化】



解答题解析

第 21 题 基础 · 解答 答案：参考程序与输出

思路：先定义类型（图纸），再创建变量并按声明顺序初始化，最后用点号逐个取成员。

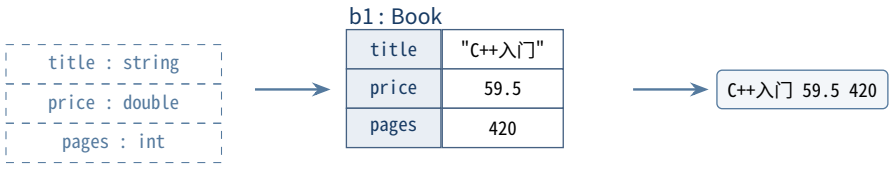
```cpp
#include <iostream>
#include <string>
using namespace std;

struct Book
{
    string title;
    double price;
    int pages;
};

int main()
{
    Book b1 = {"C++入门", 59.5, 420};
    cout << b1.title << " " << b1.price << " " << b1.pages << endl;
    return 0;
}
```

```
C++入门 59.5 420
```

【解析区图：模板 → 创建 b1 → 按顺序输出】



评分点：(1) 类型定义含三个成员且末尾有分号；(2) 初始化顺序与声明顺序一致（"C++入门"、59.5、420）；(3) 输出顺序为书名、价格、页数。

第 22 题 基础 · 解答 答案：平均分 82.33；最高分 Ann（90）

思路：`len = sizeof(arr)/sizeof(arr[0])` 得 3；循环中累加 `sum`，并用 `best` 记录当前最高分的**下标**（比较的是 `arr[i].score` 与 `arr[best].score`）。

```cpp
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Student { string name; int score; };

int main()
{
    Student arr[3] = {{"Ann", 90}, {"Bob", 75}, {"Cat", 82}};
    int len = sizeof(arr) / sizeof(arr[0]);

    int sum = 0;
    int best = 0;                       // 记录当前最高分的下标
```

```
    for (int i = 0; i < len; i++)
    {
        sum += arr[i].score;
        if (arr[i].score > arr[best].score)
            best = i;
    }
    cout << fixed << setprecision(2) << "平均分: " << (double)sum / len << endl;
    cout << "最高分: " << arr[best].name << " (" << arr[best].score << ") " << endl;
    return 0;
}
```

> 平均分: 82.33
> 最高分: Ann (90)

【解析区图：循环每一轮 sum 与 best 的变化】

i	arr[i].score	sum	best（最高分下标）
0	90	90	0
1	75	165	0
2	82	247	0

247/3 ≈ 82.33；arr[0] 是 Ann

易错：整数相除会丢小数，必须写 (double)sum / len。

第 23 题 基础 · 解答 答案：(1) 85；(2) 70

思路：要改原结构体，就传**地址**；传值只改副本。

```
#include <iostream>
#include <string>
using namespace std;

struct Student { string name; int score; };

void addScore(Student *p, int d)       // 地址传递
{
    p->score += d;
}

void addScoreByValue(Student p, int d) // 值传递（对比用）
{
    p.score += d;
}

int main()
{
    Student s = {"Dan", 70};
    addScore(&s, 15);
    cout << s.score << endl;           // 85

    Student t = {"Dan", 70};
    addScoreByValue(t, 15);
    cout << t.score << endl;           // 70
    return 0;
}
```

```
85
70
```

【解析区图：地址传递 vs 值传递】



(2) 为什么：值传递时函数里的 p.score 是 t 的副本，加到 85 后函数一结束副本就消失，t.score 仍为 70。

第 24 题　基础 · 解答　答案：先定义 Student；输出 Zhao:Sun 95

(1) 原因：编译器从上往下读。读到 Teacher 里的 Student stu; 时，Student 还没有定义，g++ 报 'Student' does not name a type。Teacher 里要放一个完整的 Student，所以**必须先让编译器知道 Student 有多大**（第 6 章“完整类型”规则）。

(2) 改正：把两个定义的**顺序对调**。

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student                // 内层类型先定义
{
    string name;
    int score;
};

struct Teacher
{
    string name;
    Student stu;              // 此时 Student 已完整
};

int main()
{
    Teacher t;
    t.name = "Zhao";
    t.stu.name = "Sun";
    t.stu.score = 95;
    cout << t.name << ":" << t.stu.name << " " << t.stu.score << endl;
    return 0;
}
```

```
Zhao:Sun 95
```

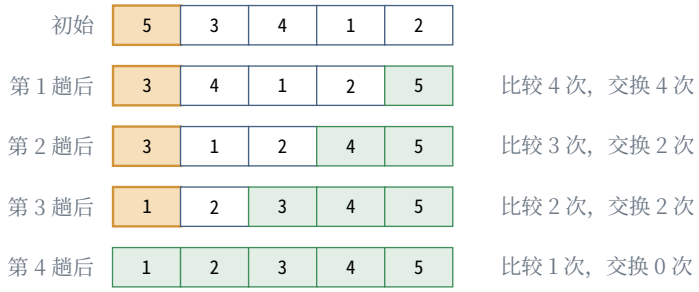
【解析区图：先后顺序决定能否编译】



第 25 题　基础 · 解答　答案：见下：比较 10 次、交换 8 次

思路：每趟把当前最大的数“冒”到末尾（绿色格）；第 i 趟（i 从 1 起）比较 $5-i$ 次。

【解析区图：每一趟结束后的数组（绿色为已就位的部分）】




```
第1趟：3 4 1 2 5
第2趟：3 1 2 4 5
第3趟：1 2 3 4 5
第4趟：1 2 3 4 5
比较10次，交换8次
```

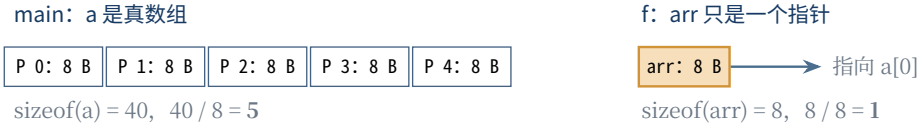
总计：比较 $4+3+2+1=10=\dfrac{5\times 4}{2}$ 次；交换次数等于初始数组的**逆序对**个数，$4+2+2+0=8$（程序 q25 已运行验证）。

11.2　提升题解析（第 26—35 题）

第 26 题　提升 · 单选　答案：B（5 1）

依据：第 6 章“数组作形参会退化成指针”。在 main 里 a 是真数组：$\text{sizeof}(a)=5\times 8=40$，$40/8=5$。传给 f 后，P arr[] 等价于 P *arr，sizeof(arr) 是**指针的大小 8**，$8/\text{sizeof}(P)=8/8=1$。

【解析区图：同一个 a，在 main 与 f 里“看起来”不一样】

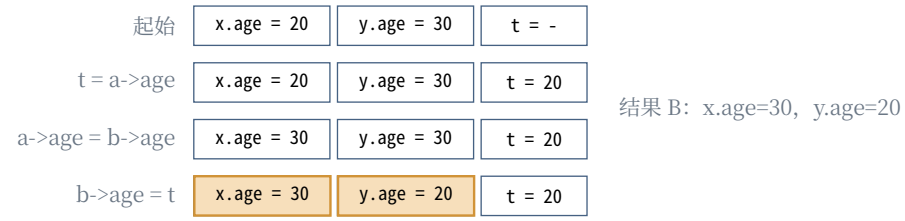


A 错：以为 arr 仍是数组。**C 错：**8 是 sizeof(arr)，题目要的是它除以 sizeof(arr[0])。**D 错：**P arr[] 作形参合法，只会有“sizeof 用在数组形参上”的警告。**平台说明：**若指针不是 8 字节（如 32 位平台为 4），结果会不同；本讲义约定 64 位平台。

第 27 题　提升 · 单选　答案：B（30 20）

依据： a、b 保存 x、y 的地址，函数里改的是原结构体，经典的“借临时变量交换”：`t=a->age; a->age=b->age; b->age=t;`。

【解析区图：过程图（每行是一步之后的值）】





A 错误 这是“没交换”的结果。若函数形参是 `Student a, Student b`（值传递），交换的只是副本，才会得到它；但这里传的是地址。



B 正确 符合上面的过程图。



C 错误 忘了用 `t`：直接 `a->age = b->age; b->age = a->age;` 才会得到 30、30（第二句读到的已是被覆盖的值）。

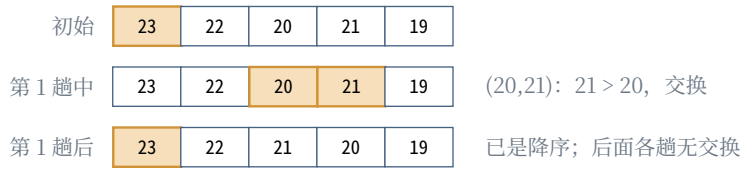


D 错误 没有任何一种步骤顺序会把 x 保持原值而让 y 也变成 20；与“交换”的含义矛盾。

第 28 题　提升 · 单选　答案：B（23 22 21 20 19）

依据： 笔记的交换条件是“后一个**大于**前一个才交换”，大的被换到前面，结果是**降序**（见第 7 章纠错 8）。第 1 趟：比较 (23,22)、(22,20)、(20,21) 交换、(20,19)，得到 23 22 21 20 19，之后各趟都不再交换。

【解析区图：过程图】





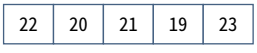
A 错误 这是**升序**。要得到它，交换条件应写成“后一个**小于**前一个”（`<`）。笔记写的是 `>`。



B 正确 降序，与过程图一致。



C 错误 这是**排序前**的顺序，说明以为程序什么也没做；但 20 与 21 满足交换条件。

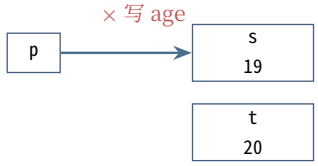


D 错误 这是“**升序**冒泡第 1 趟后”的样子（把最大的 23 冒到末尾）；混淆了比较方向。

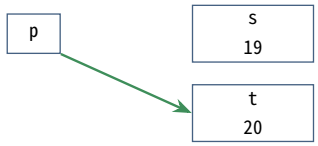
第 29 题　提升 · 多选　答案：ACD

依据：第 6 章三种 const 位置：`const T *p`（锁内容）、`T *const p`（锁指向）、`const T *const p`（两样都锁）。每条均已用 g++ 逐条验证。

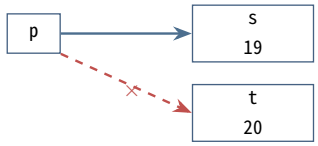
【解析区图：每个选项想做什么、被什么锁住】



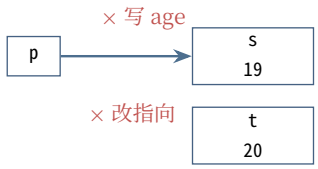
A 正确 `const Student *p`：内容被锁，`p->age = 20;` 报错（`assignment of member 'Student::age' in read- only object`）。**报错 → 选。**



B 错误 指针本身不被锁，可以改指向 `t`，能通过编译。**不报错 → 不选。**



C 正确 `Student *const p`：指向被锁，`p = &t;` 报错（`assignment of read-only variable 'p'`）。**报错 → 选。**



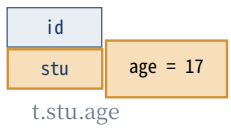
D 正确 两样都锁，`p->age = 20;` 报错。**报错 → 选。**

> **记忆办法** 看 * 的位置：const 在 * **左边** → 锁“被指的内容”；在 * **右边** → 锁“指针自己”。

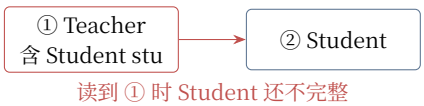
第 30 题　提升 · 多选　答案：ABD

依据：第 6 章“完整类型”规则：嵌套的成员必须是已定义完整的类型；结构体不能直接包含自己。

【解析区图：逐个选项】



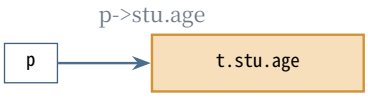
A 正确 先取 `t.stu`，再取它的 `age`，合法。



B 正确 `field 'stu' has incomplete type 'Student'`（已验证）。



C 错误 结构体里放“自己”会无限嵌套，大小无法确定，编译报 `field 's' has incomplete type`（已验证）。想表达“指向同类”应使用**指针**（链表）。

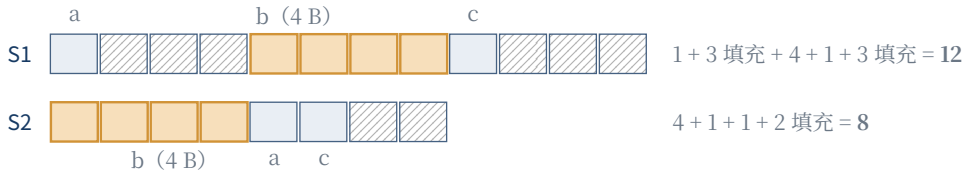


D 正确 `p->stu` 是指向的 `t` 的成员 `stu`，再 `.age`，合法。

第 31 题　提升 · 填空　答案：12；8

依据：第 4 章补全：成员按声明顺序排列，int 要放在 4 的倍数偏移处，整个结构体大小是 4 的倍数。成员完全相同，顺序不同，大小就不同。

【解析区图：逐字节布局（灰色斜线为填充字节）】

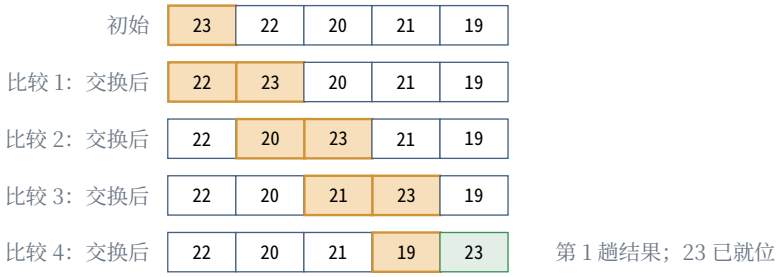


启发：把小成员放在一起，可以少填充。程序 q31 在 g++ 64 位下运行结果为 12 8。

第 32 题　提升 · 填空　答案：22 20 21 19 23；10 次

依据：第 1 趟把 23 一路“冒”到末尾：比较 (23,22)(23,20)(23,21)(23,19)，4 次比较都交换。总比较次数 $= \dfrac{5 \times 4}{2} = 10$。

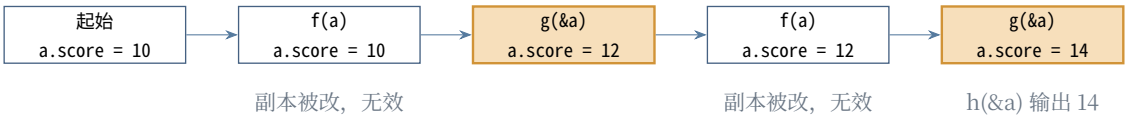
【解析区图：第 1 趟逐次比较（橙色：本次比较的相邻对）】



第 33 题　提升 · 填空　答案：14

依据：f 值传递，改的是副本，**无效**；g 地址传递，每次 +2；两次 g 共 +4；h 的 const 指针只读。$10 + 4 = 14$。

【解析区图：a.score 随调用变化】



第 34 题　提升 · 综合　答案：见下

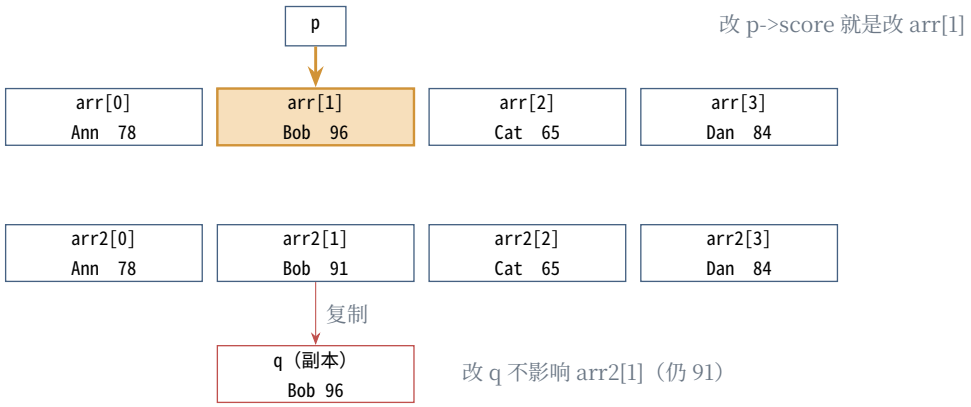
(1) 输出：

```
Bob 96
arr[1].score = 96
arr2[1].score = 91, q.score = 96
```

(2) findMax 返回的是**地址**：p 直接指向 arr[1]，p->score += 5 改的就是数组里的那个元素，所以 arr[1].score 变成 96。

(3) findMaxCopy 返回**整个结构体的副本**：q 是新变量，加 5 只改 q，arr2[1] 不受影响，仍是 91。

【解析区图：题意 → 过程 → 结果（上：返回指针；下：返回副本）】



> **易错：不要返回局部变量的地址** `findMax` 返回的指针指向**数组里的元素**，数组在 `main` 里还活着，所以安全。如果函数返回的是指向**函数内部局部变量**的指针，函数一结束该变量就消失，指针就“悬空”了。

变式思考：若 `n` 可能为 0，`&a[0]` 就越界了；应先判断 `if (n <= 0) return nullptr;`，调用方使用前也要检查 `p != nullptr`。

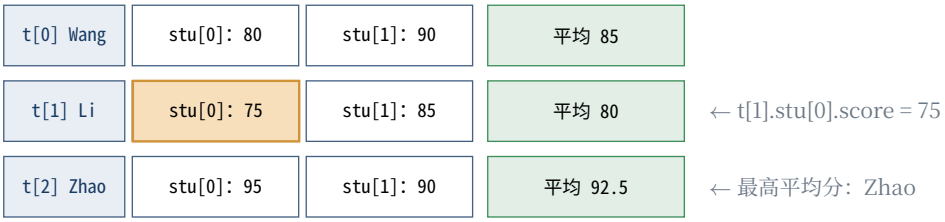
第 35 题　提升 · 综合　答案：见下

输出：

```
75
Wang 平均分 85
Li 平均分 80
Zhao 平均分 92.5
最高：Zhao
```

(1) `t[1].stu[0].score`：先选第 2 位老师（下标 1），再选他的第 1 名学生（下标 0），取成绩，是 75。**(2)** 每位老师两名学生取平均：$(80+90)/2=85$，$(75+85)/2=80$，$(95+90)/2=92.5$。`average` 的形参是 `const Teacher *`，只读不改；除以 `2.0` 保证得到小数。**(3)** 用 `best` 记录最高平均分的老师下标，最后是 Zhao。

【解析区图：访问路径与平均分】



变式思考：若要求“某位老师所带学生中的最高分”，把标号 (2) 处改成对 `stu` 的内层循环：`int mx = t[i].stu[0].score; for (int j = 1; j < 2; j++) if (t[i].stu[j].score > mx) mx = t[i].stu[j].score;`，思路与第 22 题相同。

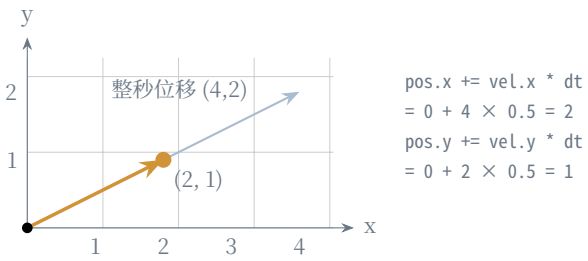
11.3 跨学科题解析（第 36—38 题）

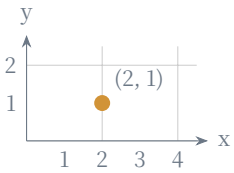
> **跨学科题的读法** 每题解析后面都有一个紫色框，说明：本题用到的**当前专题知识**、借用的**外部情境**、**为什么不算超纲**、可以带走的**迁移点**。

第 36 题 跨学科 · 单选 答案：A，即 $(2,1)$

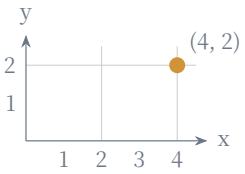
过程： `q.pos` 起点 $(0,0)$；速度 $(4,2)$、$\Delta t = 0.5$，每个分量的位移为 $4 \times 0.5 = 2$、$2 \times 0.5 = 1$；`q.pos.x += …` 即 $0+2=2$，`q.pos.y` 即 $0+1=1$。访问用的是两层点号：`q.pos.x`、`q.vel.y`（第 6 章嵌套）。

【解析区图：题意 → 过程 → 结果】

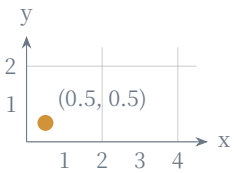




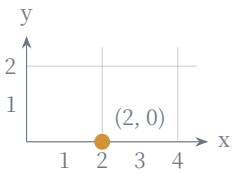
A 正确 与过程一致。



B 错误 忘了乘 `dt`：这是一**整秒**的位移，而 `dt = 0.5`。



C 错误 把 `dt` 当成了位移加到位置上；实际是 `vel * dt`。

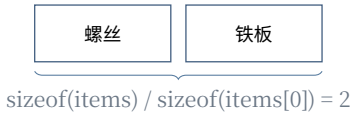


D 错误 漏了 y 方向：程序里 `q.pos.y += …` 也执行了，y 变成 1。

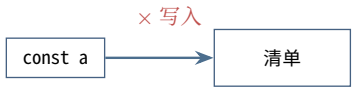
> **跨学科说明** **当前专题知识：** 嵌套结构体（`Particle` 里含 `Vec2`）、两层点号访问、复合赋值。
> **借用的外部情境：** 质点的“位置 += 速度 × 时间”。
> **为什么不算超纲：** 物理公式已在背景里写明，只需做两次乘加。
> **迁移点：** 任何“一个对象由几组同类数据组成”的问题（游戏角色的位置与速度、地图上的坐标点）都可以套用“嵌套结构体 + 逐层点号”。

第 37 题　跨学科 · 多选　答案：ABD

【解析区图：逐个选项】



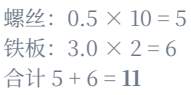
A 正确 items 是真数组，两个元素的字节数相除得 2（第 5 章公式）。



B 正确 const 保证函数里不能通过 a 改清单（第 6 章）。



C 错误 形参 a 只是指针（退化）。sizeof(a) 是 8，sizeof(a[0]) 即 sizeof(Item)，在本机为 48，整数除法得 **0**，不是 2（第 26 题同理）。所以数组长度要作为参数 n 传入。



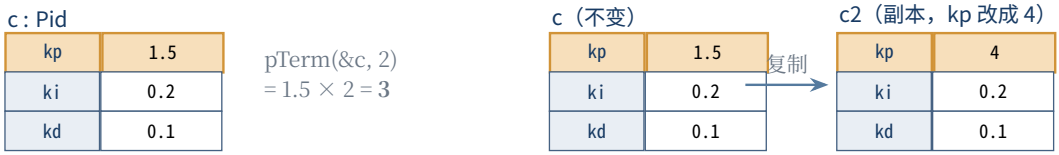
D 正确 与程序输出 11 一致（已运行验证）。

跨学科说明 **当前专题知识：**结构体数组、数组作形参会退化成指针（要另传长度 n）、const 指针只读、成员访问 a[i].weight。
借用的外部情境：仓库清单：总重量 = 单件重量 × 件数后求和。
为什么不算超纲：背景里给了公式，只涉及乘法与加法。
迁移点：“表格式数据 + 逐行汇总”（订单、成绩单、传感器采样表）都是“结构体数组 + 循环 + 把长度传进函数”的套路。

第 38 题　跨学科 · 填空　答案：第一行 3；第二行 1.5 4

过程：pTerm(&c, 2) 返回 $K_p \times e = 1.5 \times 2 = 3$。Pid c2 = c; 复制出独立的 c2；把 c2.kp 改成 4，c.kp 仍是 1.5。pTerm 的形参 const Pid * 保证函数不会改控制参数。

【解析区图：题意 → 过程 → 结果】



跨学科说明 **当前专题知识：**结构体变量整体复制后相互独立、const 指针传参、-> 访问成员。
借用的外部情境：控制参数 K_p, K_i, K_d 与“比例项 $= K_p e$”。
为什么不算超纲：背景给出全部公式，只用一次乘法，无需自动控制知识。
迁移点：“一组配置打包成结构体、复制后调参而不影响原配置”适用于任何需要保存多套参数的程序。

第 12 章　参考资料与说明

一、原始教学依据

本讲义的主体内容来自你提供的 13 页手写笔记（Memo No. 417—429，2026/10/2，第八章“结构体”），以及随笔记提供的《讲义写作规范》写作规范。笔记中的代码均已逐段编译运行；笔记里无法辨认或有明显笔误之处，已在第 3 章以“原始资料此处无法可靠辨认”和“原图纠错”标出。

二、核心定义与规则的核验来源

- cppreference，Aggregate initialization（聚合初始化：按声明顺序、未给出的成员值初始化、缩窄转换为错误）：
  `en.cppreference.com/w/cpp/language/aggregate_initialization`
- cppreference，Member access operators（.、->、* 的含义；对内建指针 `E1->E2` 等价于 `(*E1).E2`）：
  `en.cppreference.com/w/cpp/language/operator_member_access`
- cppreference，Objects and alignment（C 语言中对象的大小、对齐与填充的概念）：
  `en.cppreference.com/w/c/language/object`

关于优先级、`const` 位置、完整类型、数组退化、对齐填充的**具体报错信息和数值结果**，全部以 g++（`-std=c++17`，64 位 Linux）**实际编译运行**的结果为准；`sizeof` 的数值依赖平台，已在相应题目中注明。

三、学科小史与科普的参考

- N. Wirth，Recollections about the development of Pascal（Pascal 与记录类型的由来）：
  `pascal.hansotten.com` 上的转载页面
- Wikipedia，C++（C with Classes 至 C++98 的时间线）：
  `en.wikipedia.org/wiki/C++`
- Wikipedia，Record (computer science)（“记录/结构体”在 COBOL、Pascal、C 中的出现）：
  `en.wikipedia.org/wiki/Record_(computer_science)`

学科史部分只写了有把握的年代与事实（如 C++ 首个商业发行版 1985 年、标准 C++98），不确定的细节一律用“约”“前后”表述，未虚构任何书号或论文编号。

四、本讲义的“教学补全”约定

凡标注“教学补全”的内容（如 `sizeof` 与对齐、数组名与指针、空指针、聚合初始化细节）都**不在你的原笔记中**，是为了让初学者读懂而加的，并与笔记原文用青绿色框明确分隔。习题（共 38 题）全部是新编题，答案均经程序或手工推演验证。

五、练习图解专项终检（38 题逐题）

题号	适合图解	题目区图	解析区图	选项小图（组数）	不适配原因
1	否	–	–	–	概念选择，无可画对象
2	否	–	–	–	纯语法辨认
3	是	是	是	–	
4	是	是	是	–	
5	是	是	是	–	
6	是	是（含选项图）	是（含选项图）	4 + 4	
7	否	–	–	–	符号等价变换，无空间关系
8	是	是	是	–	
9	是	是	是	–	
10	否	–	–	–	语义概念（const 含义），由第 29 题图解承担
11	否	–	–	–	概念陈述判断
12	是	是	是（含选项图）	4	
13	是	是	是（含选项图）	4	
14	是	是	是	–	
15	是	是	是（含选项图）	4	
16	否	–	–	–	关键词填空
17	是	是	是	–	
18	是	是	是	–	
19	是	是	是	–	
20	是	是	是	–	
21	是	是	是	–	
22	是	是	是	–	
23	是	是	是	–	
24	是	是	是	–	
25	是	是	是	–	
26	是	是	是	–	
27	是	是（含选项图）	是（含选项图）	4 + 4	
28	是	是（含选项图）	是（含选项图）	4 + 4	
29	是	是	是（含选项图）	4	
30	是	是	是（含选项图）	4	
31	是	是	是	–	
32	是	是	是	–	
33	是	是	是	–	
34	是	是	是	–	
35	是	是	是	–	
36	是	是（含选项图）	是（含选项图）	4 + 4	
37	是	是	是（含选项图）	4	
38	是	是	是	–	

覆盖统计：适合图解的题 32/38，其中题目区图 32/32，解析区图 32/32；有可画选项的单选/多选题 11 道（第 6、12—15、27—30、36、37 题），第 6、27、28、36 题题目区和解析区均有选项级小图，第 12、13、15、29、30、37 题解析区逐选项配小图，第 14 题用一幅图同时呈现四种写法的等价关系。不适配的 6 题（第 1、2、7、10、11、16 题）已写明原因，没有硬配装饰图。